technische universität
dortmund

Bachelorarbeit

Realisierung einer Game-Engine in eingebetteten Echtzeitbetriebssystemen

Maximilian Adolenko
24. August 2026

Supervisors:
Dr.-Ing. Reinhard Georg von der Brüggen
Kay Heider, M.Sc.

Technische Universität Dortmund
Fakultät für Informatik
Lehrstuhl Informatik 12 (Eingebettete Systeme)
Design Automation for Embedded Systems Group
`https://daes.cs.tu-dortmund.de`

Acknowledgements

Ich bedanke mich bei Kay Heider und Nils Hölscher für tatenkräftigen Unterstützung und Beratung bei dem Umsetzen dieser Arbeit und für das Bereitstellen der gesamten Hardware.

Inhaltsverzeichnis

1 Einleitung **1**

1.1 Struktur der Arbeit 1

1.2 Verwandte Arbeiten 1

2 Grundlagen **3**

2.1 Echtzeitbetriebssysteme 3

2.1.1 Was ist ein Echtzeitbetriebssystem? 3

2.1.2 Tasks 3

2.1.3 Interrupts 4

2.1.4 Queues 4

2.2 Was ist FreeRTOS? 4

2.3 Game-Engines 5

3 Design **7**

3.1 Anforderungen 7

3.2 Architektur-Design der Game-Engine 8

3.3 Hardware-Ebene 10

3.4 Game-Engine-Kern 10

3.5 User-Ebene 13

4 Umsetzung der Hardware **15**

4.1 Verwendete Bauteile 15

4.1.1 Microcontroller 15

4.1.2 Bildschirm 15

4.1.3 Eingabe 16

4.1.4 Audio 16

4.2 Aufbau des Systems 16

4.3 Praktische Umsetzung 18

5 Umsetzung der Software **21**

5.1 Eingabe 21

5.1.1 Anbindung der physischen Knöpfe 21

5.1.2 ISR 22

5.1.3 Input-Service . . . 23
5.2 Logic-Task . . . 24
5.3 Render-Task und Grafik . . . 26
5.3.1 Render-Task . . . 26
5.3.2 TileMap . . . 28
5.3.3 Texturen . . . 28
5.4 Timing . . . 29
5.5 Audio . . . 29
5.5.1 Töne und Soundeffekte . . . 30
5.5.2 Audio-Service . . . 30
5.5.3 Audio-Task . . . 31
5.6 Dynamische Framerateanpassung . . . 32
5.6.1 Ausführungszeiten . . . 32
5.6.2 Deadline-Misses . . . 33
5.6.3 Transfer-Misses . . . 33
5.6.4 Statistik-Task . . . 34
5.7 Schnittstelle zur Spieleprogrammierung . . . 36
5.8 Spieleobjekte . . . 37
5.8.1 Kollision . . . 37
5.8.2 Physik . . . 38
5.8.3 Bewegung und Dirty-Tile-Markierung . . . 38
5.9 Init-Task . . . 39

6 Proof of Concept **41**
6.1 Flatter Vogel . . . 41

7 Evaluation **43**
7.1 Evaluations-Setup . . . 43
7.2 Task-Ansatz vs. Main-Loop . . . 43
7.2.1 Messergebnisse . . . 44
7.2.2 Auswertung . . . 46
7.3 Statische und dynamische Framerates . . . 46
7.3.1 Messergebnisse . . . 47
7.3.2 Auswertung . . . 50
7.4 Optimale Tilegröße . . . 50
7.4.1 Messergebnisse . . . 51
7.4.2 Auswertung . . . 52

8 Zusammenfassung **53**
8.1 Was wurde erreicht? . . . 53

8.2 Ausblick für die Zukunft 53

Abbildungsverzeichnis **55**

Algorithmenverzeichnis **57**

Quellcodeverzeichnis **59**

Literatur **61**

Eidesstattliche Versicherung **63**

1 Einleitung

Üblicherweise werden Echtzeitbetriebssysteme in kritischen Systemen wie dem autonomen Fahren verwendet, da diese ein deterministisches und vorhersehbares Zeitverhalten bieten. Aber auch Game-Engines würden von solch einem deterministischem Zeitverhalten profitieren, da damit zum Beispiel eine konstante Framerate erreicht werden kann oder zeitnah auf Eingaben reagiert werden kann.

Im Rahmen dieser Bachelorarbeit soll eine Game-Engine mit Hilfe von Strukturen eines Echtzeitbetriebssystems entwickelt werden. Diese soll ermöglichen, dass unterschiedliche Spiele entwickelt werden können, ohne dass die Grundstrukturen von Spielen, wie zum Beispiel die Eingabeabfrage oder Rendering, immer wieder neu implementiert werden müssen. Außerdem soll im Rahmen dieser Arbeit ein physischer Handheld entwickelt werden, auf dem mit der Game-Engine entwickelte Spiele ausgeführt und gespielt werden können. Außerdem wird in dieser Arbeit als Ziel gesetzt, eine dynamische Framerate-Anpassung innerhalb der Game-Engine umzusetzen.

1.1 Struktur der Arbeit

Zu Beginn werden in Kapitel 2 die nötigen Grundlagen dargestellt. Daraufhin wird in Kapitel 3 das Design der Architektur der Game-Engine vorgestellt. Kapitel 4 und 5 stellen die Umsetzung der Architektur dar. Zum Abschluss der Arbeit wird in Kapitel 6 ein mit der Game-Engine entwickeltes Spiel vorgestellt und in Kapitel 7 wird die umgesetzte Game-Engine evaluiert.

1.2 Verwandte Arbeiten

Viele verwandte Arbeiten in der Game-Engine-Entwicklung mit Echtzeitbetriebssystemen gibt es nicht. Einige Spiele wurden mit Hilfe von Echtzeitbetriebssystemen auf Microcontrollern entwickelt. Beispielsweise wurde das Spiel Snake für den STM32 [22] oder ein LED-Muster Spiel für den ESP32, in dem man LED-Muster replizieren muss [15], mit Hilfe von Echtzeitbetriebssystemen entwickelt. Hierbei handelt es sich aber um manuelle Implementierungen, ohne dass Features für die Entwicklung durch eine Game-Engine bereitgestellt werden.

2 Grundlagen

Im Folgendem werden Grundlagen erläutert, die für die Erstellung dieser Arbeit erarbeitet wurden und dem Verständnis dieser Arbeit helfen.

2.1 Echtzeitbetriebssysteme

In diesem Abschnitt werden die Kerngrundlagen von Echtzeitbetriebssystemen erläutert.

2.1.1 Was ist ein Echtzeitbetriebssystem?

Bei Echtzeitbetriebssystemen handelt es sich um Programme, welche Ressourcen verwalten und die zeitliche Ausführung von Tasks koordinieren. Eine Kerneigenschaft von Echtzeitbetriebssystemen ist, dass die Koordinierung der Tasks vorhersehbar und deterministisch abläuft, da Echtzeitbetriebssysteme in Echtzeit-Anwendungen genutzt werden [16].
Die genauen bereitgestellten Funktionen eines Echtzeitbetriebssystems unterscheiden sich je nach Anwendungsfall. Dennoch verfügen alle Echtzeitbetriebssysteme über einen Kernel als Grundbaustein. Der Kernel besteht aus einem Scheduler, Objekten und Services [16].
Der Scheduler in einem Echtzeitbetriebssystem koordiniert die zeitliche Ausführung von Tasks. Um zu bestimmen, welche Task aktuell Rechenzeit bekommt, werden unterschiedliche Scheduling-Algorithmen genutzt. Diese nutzen unterschiedliche Eigenschaften der Tasks, um die Rechenzeit zu verteilen. Kernel-Objekte werden bereitgestellt, damit sie als Grundbaustein von Entwickler genutzt werden können, um Echtzeit-Anwendungen zu erstellen. Services dienen als Schnittstelle, um die Funktionen eines Echtzeitbetriebssystems nutzen zu können [16].

2.1.2 Tasks

Echtzeit-Anwendungen werden normalerweise in Richtung Parallelität konzepiert. Um dies zu erreichen werden Tasks genutzt, wobei es sich um unabhängige Ausführungseinheiten handelt [16].
Tasks besitzen unterschiedliche Eigenschaften. Zu den Eigenschaften von Tasks gehört beispielsweise, dass diesen eine Priorität [16] zugeordnet wird und dass Tasks bis zu einem bestimmten Zeitpunkt, einer so genannten Deadline, ausgeführt werden müssen [3].
Je nach genauem Scheduling-Algorithmus entscheidet der Scheduler anhand dieser Eigenschaften, welche Tasks ausgeführt werden dürfen. Zum Beispiel wird bei dem „Preemptive

Priority-Based Scheduling“ immer die Task mit der höchsten Priorität ausgeführt [16]. Bei dem Scheduling-Algorithmus „Earliest Deadline First“ wird die Task mit der frühesten absoluten Deadline ausgeführt [3]. Tasks können in zwei unterschiedliche Kategorien bezüglich der Echtzeitanforderung eingeordnet werden. Harte Deadlines bedeutet, dass es zu schlimmen Folgen kommen kann, wenn eine Deadline verpasst wird. Bei soften Deadlines kommt es zu keinen schlimmen Folgen, wird die Deadline nicht eingehalten, jedoch fühlt sich die Anwendung dadurch zum Beispiel träge an [1]. Außerdem unterscheiden sich Tasks in ihrem Ausführungsverhalten. Tasks können einmalig oder in einer unendlichen Schleife laufen. Tasks, die unendlich laufen, müssen einen blockierenden Aufruf haben, damit andere Tasks ausgeführt werden können [16]. Endlose Tasks können hierbei periodisch oder aperiodisch ausgeführt werden. Periodische Tasks werden immer in festen Perioden aktiviert, aperiodische Tasks haben hingegen keine zeitliche Konsistenz in ihrem Ausführungsverhalten. Aperiodische Tasks mit einem Minimum an zeitlichem Abstand werden sporadische Tasks genannt [16]. Eine Ausführung einer Task wird auch Job genannt [3].

2.1.3 Interrupts

Bei Interrupts handelt es sich um Signale, die beispielsweise durch Hardware an den Prozessor geschickt werden. Erhält der Prozessor solch ein Signal, stoppt dieser seine aktuelle Aufgabe und behandelt diesen Interrupt durch eine Interrupt Service Routine (ISR) [21].

2.1.4 Queues

Queues werden dafür genutzt, dass Tasks miteinander kommunizieren und Daten synchronisieren können. Hierbei können Nachrichten von Tasks in die Queue gelegt und aus der Queue gelesen werden. Nachrichten werden nach dem First-In-First-Out-Prinzip innerhalb von Queues verwaltet [16].

2.2 Was ist FreeRTOS?

Bei FreeRTOS handelt es sich um ein Echtzeitbetriebssystem, welches genutzt wird, um Echtzeit-Anwendung für Microcontroller und -prozessoren zu entwickeln. FreeRTOS setzt die vorgestellten Funktionalitäten eines Echtzeitbetriebssystems praktisch um. Der FreeRTOS Scheduler nutzt den Scheduling-Algorithmus „Fixed Priority Preemptive Scheduling with Time Slicing“. Dieser führt immer die Task mit der höchsten Priorität aus, die ausgeführt werden kann. Können mehrere Tasks mit der gleichen Priorität ausgeführt werden, wird unter diesen die Ausführungszeit gleich verteilt. Preemption, also das Verdrängen der aktuell ausgeführten Task, und das Time Slicing können deaktiviert werden [1].

2.3 Game-Engines

Bei Game-Engines handelt es sich um Software, welche als Grundlage für die Entwicklung von Spielen genutzt werden kann. Game-Engines trennen das eigentliche Spiel von Komponenten, welche immer wieder in Spielen vorkommen. Somit können diese wiederverwendet werden und müssen für das Entwickeln von Spielen nicht wieder neu entwickelt werden. Zu den Komponenten einer Game-Engine gehören beispielsweise das Rendering, Audiobereitstellung, Eingabebereitstellung, Verwaltung von Game Assets wie Texturen oder Sound, Bereitstellung von Physik und Kollisionsüberprüfung, Animationen und Logikausführung. Die genaue Trennung zwischen Game-Engine und Spiel unterscheidet sich je Game-Engine, somit können Game-Engines allgemeiner gehalten sein oder auf bestimmte Spiele-Arten getrimmt sein [11].

3 Design

Bevor die Game-Engine mit Hilfe des eingebetteten Echtzeitbetriebssystems entwickelt werden kann, muss diese designt werden. Dafür werden im Folgenden die Anforderungen der Game-Engine untersucht und die daraus resultierende Architektur erläutert.

3.1 Anforderungen

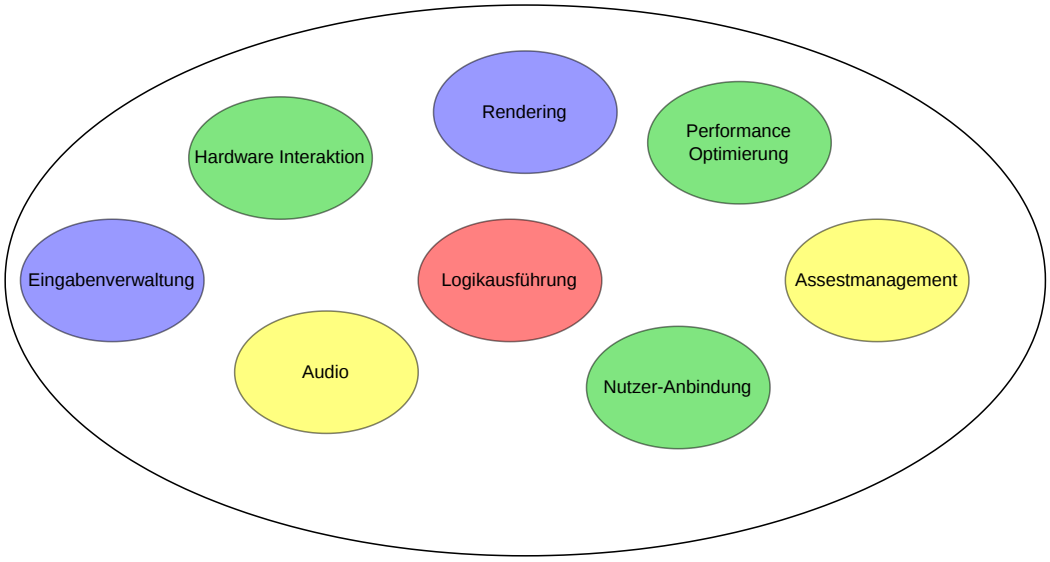


Abbildung 3.1: Komponenten einer Game-Engine

Wie in Kapitel 2 beschrieben, übernimmt eine Game-Engine unterschiedliche Aufgaben, welche im Hintergrund laufen, wie z.B. das Rendering, die Inputverarbeitung oder die Audiosteuerung, aber auch Aufgaben, die den Nutzer direkt betreffen, wie z.B. die Bereitstellung von Kollisionsprüfung, Physik, die Möglichkeit, Spielelogik zu programmieren oder Assets zu verwalten. Abbildung 3.1 stellt mögliche Komponenten einer Game-Engine dar.

Daher muss die in eingebetteten Echtzeitbetriebssystemen umgesetzte Game-Engine einige Anforderungen erfüllen, um die Komponenten einer Game-Engine nachzubilden bzw. bereitzustellen.

Es muss ein Hardwaresystem vorhanden sein, worauf die Game-Engine ausgeführt werden kann und welches als „Spielsystem“ von dem Nutzer verwendet werden kann.
Die Game-Engine muss mit der Hardware kommunizieren können, um z.b. Bilddaten verschicken zu können, Eingaben lesen zu können oder die Audio-Ausgabe steuern zu können.
Die Game-Engine muss Eingaben verarbeiten können und diese dem Nutzer als Eingabeevents zur Nutzung in der Spielelogik zur Verfügung stellen. Die Regelung der Kommunikation kann hierbei durch das Echtzeitbetriebssystem beispielsweise durch Interrupts übernommen werden.
Außerdem muss die Game-Engine die Spielelogik ausführen können und sie muss automatisch aus dem aktuellen Spielzustand das Bild rendern können. Zudem müssen die beiden Prozesse in einem gleichmäßigen Takt laufen, um ein flüssiges Bild und eine gleichmäßige Logikausführung und Inputverarbeitung zu ermöglichen und im Endeffekt eine stabile Framerate zu erhalten.
Außerdem muss die Game-Engine regelmäßig und gleichmäßig den aktuellen Sound updaten, um eine unterbrechungsfreie Audio-Ausgabe zu ermöglichen. Die Prozesse könnten beispielsweise als Tasks implementiert werden.
Um die Framerate dynamisch anpassen zu können, muss die Game-Engine Performance-Daten sammeln, diese verarbeiten und nach bestimmten Regeln entscheiden, ob die Framerate und somit der Takt runtergesetzt oder hochgesetzt wird. Hierbei bieten Echtzeitbetriebssysteme durch z.B. Deadline-Misses passende Indikatoren an, um zu überprüfen, ob das System überlastet ist oder nicht.
Außerdem muss die Game-Engine eine Schnittstelle für den Nutzer haben, wo dieser die Spielelogik implementieren oder Spieledaten, wie Texturen oder Soundeffekte, speichern kann. Zudem soll die Game-Engine dem Nutzer die Möglichkeit geben, Spieleszenen grafisch zu designen und „Spielobjekte“ zu verwalten.
Die Game-Engine soll dem Nutzer auch die Möglichkeit geben, Kollisionsüberprüfungen automatisch zu tätigen, ohne die Kollisionslogik jedes Mal selber implementieren zu müssen und ein paar simple Physik-Eigenschaften sollen dem Nutzer zur Verfügung stehen.

3.2 Architektur-Design der Game-Engine

Um die Game-Engine in eingebetteten Echtzeitbetriebssystemen umzusetzen, eignet sich folgendes Design, welches sich in drei Schichten aufteilen lässt. Diese 3 Schichten werden im Folgenden genauer beschrieben und begründet. In Abbildung 3.2 wird die gesamte Architektur der Game-Engine dargestellt.

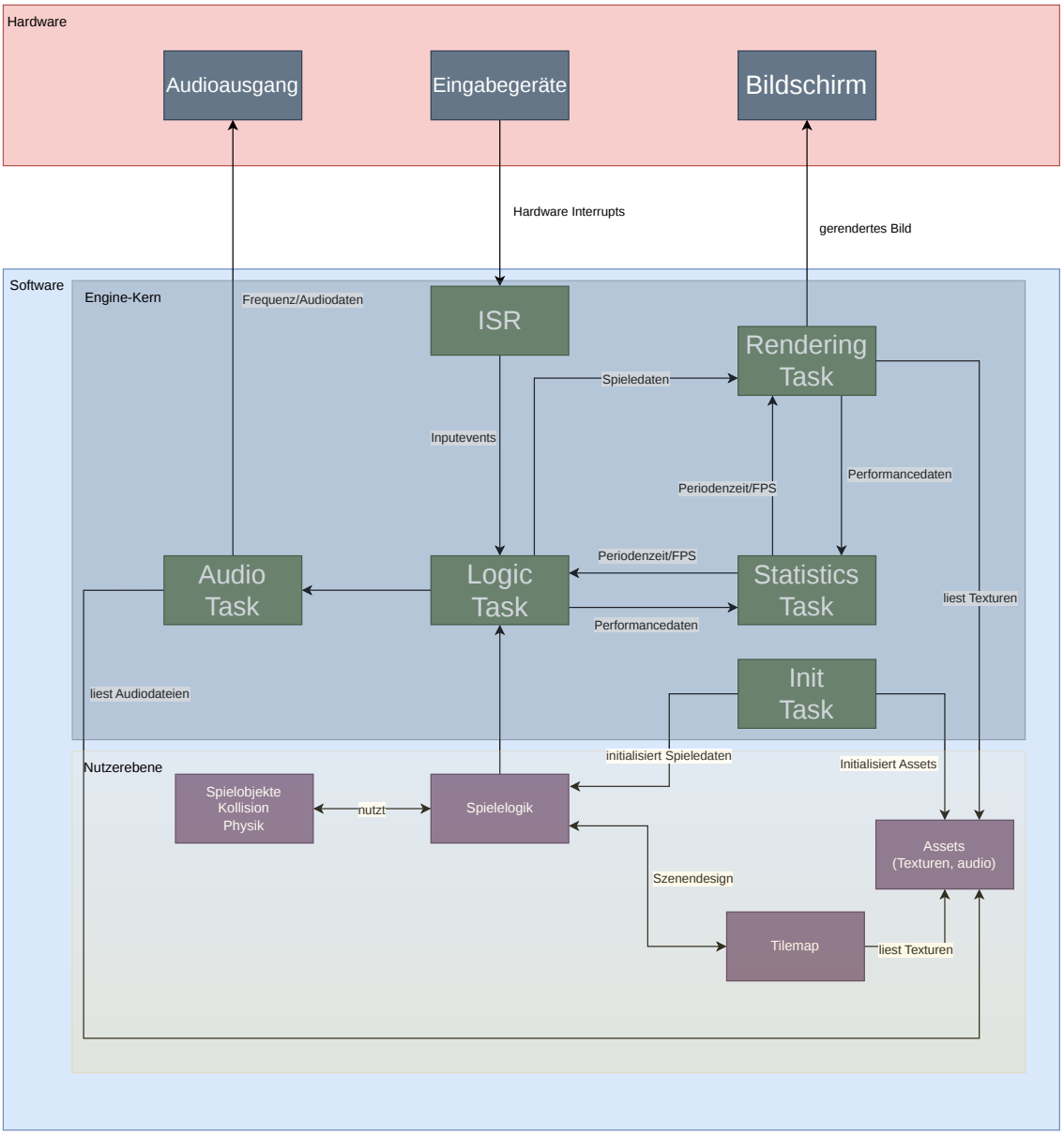


Abbildung 3.2: Architektur der Game-Engine

3.3 Hardware-Ebene

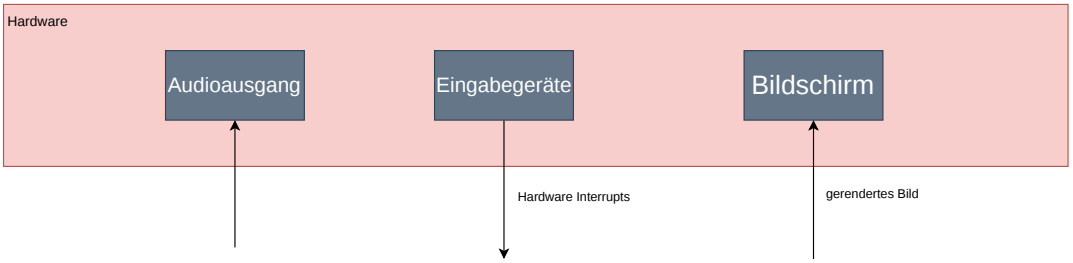


Abbildung 3.3: Architektur der Hardware-Schicht

Die Hardware-Schicht, dargestellt in Abbildung 3.3, bildet die Grundlage der Game-Engine, da sie das eingebettete System widerspiegelt, worauf die im Echtzeitbetriebssystem umgesetzte Game-Engine ausgeführt wird. Damit der Hardwareaufbau auch als Spielesystem nutzbar ist, wird ein Handheld-ähnlicher Aufbau verfolgt.
Um das Spiel grafisch darzustellen, benötigt man einen Bildschirm. Damit der Nutzer mit dem System und dem darauf laufenden Spiel interagieren kann, benötigt die Hardware auch Eingabemöglichkeiten. Um die Anforderung, dass Audioeffekte genutzt werden können, zu erfüllen, muss die Hardwareebene auch einen Audioausgang bereitstellen. Für die Steuerung der Hardwarekomponenten und die Ausführung der Game-Engine soll ein Microcontroller genutzt werden. Dieser ist jedoch nicht direkt in 3.3 vermerkt.
Um möglichst kleine Latenzen bei Datenzugriffen, wie z.B. bei dem Lesen von Texturen oder Audiodateien, zu haben, wird auf ein externes Speichermedium wie zum Beispiel eine SD-Karte verzichtet. Daher muss bei der Umsetzung darauf geachtet werden, einen Microcontroller mit möglichst viel internem Speicherplatz zu wählen. Möglichst kleine Datenlatenzen sind für die Game-Engine wichtig, damit die Ausführungszeiten einzelner Tasks nicht durch lange Datenzugriffszeiten in die Höhe getrieben werden. Kleinere Ausführungszeiten führen dementsprechend zu größeren Framerates, was ein flüssigeres Spieleerlebnis ermöglicht.

3.4 Game-Engine-Kern

Um die softwarebezogenen Anforderungen zu erfüllen, wird die Game-Engine in 2 Schichten aufgeteilt. Bei dem „Engine-Kern“ , abgebildet durch Abbildung 3.4, handelt es sich um den Teil der Game-Engine, mit dem der Nutzer nicht direkt interagieren muss. Das heißt, dieser Teil der Software übernimmt alle Aufgaben im Hintergrund der Game-Engine, wozu die Logikausführung, das Rendering, Audiorendering, Inputverarbeitung und dyna-

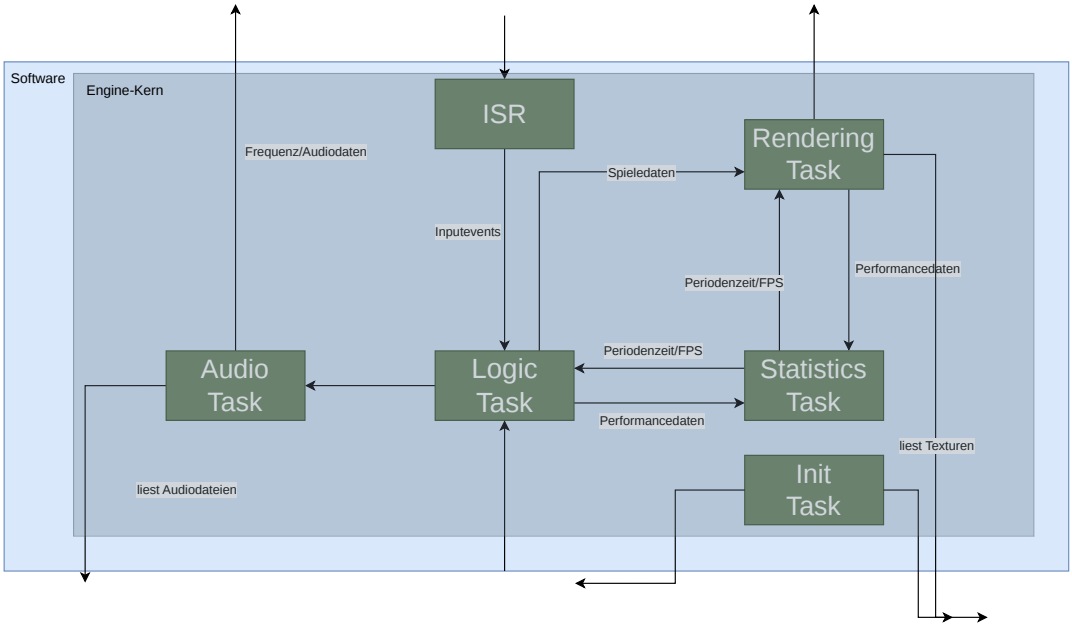


Abbildung 3.4: Architektur des Engine-Kerns

mische Framerate-Anpassung gehören. Um diesen Teil der Software umzusetzen, eignen sich besonders Konzepte aus Echtzeitbetriebssystemen.

Eingaben

Um die Inputverarbeitung umzusetzen, bieten sich Hardware-Interrupts an, welche beim Betätigen der Eingabegeräte ausgelöst werden. Interrupts eignen sich besonders, da sie eine möglichst zeitnahe Bearbeitung des Inputs ermöglichen. Außerdem wird im Vergleich zu Polling, wo aktiv nach dem Inputzustand gefragt wird, kaum CPU-Zeit verbraucht [21]. Softwareseitig soll die Verarbeitung der Interrupts durch eine ISR umgesetzt werden, welcher das anliegende Signal am Eingabegerät bei einem Interrupt zu bestimmten Eingabeereignissen verarbeiten soll und dieses dann an die Logic-Task schickt.

Tasks

Um Logikausführung, Audioverarbeitung, Rendering und dynamische Framerate-Anpassung umzusetzen, eignet sich das Konzept der Tasks aus Echtzeitbetriebssystemen. Tasks eignen sich für diese Aufgaben aus unterschiedlichen Gründen.
Zum einen ermöglichen Tasks, dass jeder Aufgabenbereich entkoppelt vom anderen ausgeführt werden kann und somit vermischen sich die Verantwortlichkeiten der einzelnen Aufgaben nicht. Zum anderen ermöglichen Tasks ein deterministisches Ausführungsverhalten,

da man diese mit festen Periodenzeiten ausführen lassen kann. Um Tasks periodisch ausführen lassen zu können, bieten Echtzeitbetriebssysteme einen zentralen Scheduler. Diesen muss man somit nicht selbst entwickeln und man hat eine Umgebung, die genau für sich wiederholende Aufgaben mit Zeitanforderungen konzipiert ist. Dies ist besonders wichtig und praktisch für die Game-Engine, da dadurch eine konstante Framerate erreicht werden kann, sodass Spiele basierend auf der Game-Engine möglichst konsistent und vorhersagbar ausgeführt werden können und zeitnah und zuverlässig auf Eingaben reagieren können. Außerdem eignet sich das deterministische Ausführungsverhalten für die Audioverarbeitung, da hier auch durch eine möglichst klein gewählte Periodenzeit das Abspielen von Soundeffekten möglichst sauber ermöglicht werden kann.

Die Logikausführung wird durch eine Task übernommen, welche die in 3.5 beschriebene, von dem Nutzer implementierte, Spielelogik ausführen soll. Außerdem wird die Logiktask von dem ISR mit Eingabeereignissen versorgt, welche dann von der Logic-Task zu den von der Game-Engine unterstützten Eingabearten umgewandelt werden, damit die Spielelogik auf diese reagieren kann. Zudem soll die Logic-Task nach jeder Ausführung sowohl den aktuellen Spielezustand einfrieren und an die Render-Task verschicken, als auch mögliche Audiobefehle an die Audio-Task verschicken, um die aktuelle Audioausgabe basierend auf Spielereignissen steuern zu können. Hierbei ist es sinnvoll, für das Rendering den Spielzustand einzufrieren, damit es nicht zu Race Conditions um die Spieledaten kommt, falls die Logic-Task gleichzeitig schon den nächsten Spielzustand berechnet, obwohl noch der alte Zustand gerendert wird.

Die Render-Task übernimmt die Bildgenerierung in der Game-Engine. Hierfür konsumiert die Render-Task den von der Logic-Task eingefrorenen Spielzustand und erzeugt daraus einen fertigen Frame, welcher dann zum Anzeigen an den Bildschirm geschickt wird. Diese beiden Tasks werden gemeinsam mit der gleichen Periodenzeit betrieben, um die konstante Framerate zu erreichen. Die Audio-Task sorgt für eine konstante Audioverarbeitung. Diese nimmt Audiobefehle von der Logic-Task entgegen, um den aktuellen Sound einzustellen oder stummzustellen und verarbeitet basierend auf der vergangenen Zeit zwischen jeder Ausführung der Task den aktuellen Ton, der gespielt werden muss. Um ein möglichst sauberes Audioerlebnis zu bekommen, muss die Periodenzeit der Audio-Task möglichst gering sein.

Um die dynamische Framerate-Anpassung umzusetzen, wird noch eine Statistik-Task angelegt. Hierbei versorgen die Logic-Task und die Render-Task jeweils diese Task mit Performancedaten. Basierend auf diesen Performancedaten entscheidet die Statistik-Task, ob die Framerate hochgesetzt, runtergesetzt oder nicht verändert werden soll. Kommt es zu

Änderungen der Framerate, wird die neue Framerate der jeweiligen Task geschickt, damit sie deren Periodenzeiten anpassen können.

Die Init-Task dient nur zur Initialisierung aller Daten und hat im weiteren Ausführungsverlauf keinen Einfluss auf die Game-Engine, da diese nur einmalig ausgeführt werden soll.

Für die Kommunikation zwischen den Tasks und für das Versenden von Daten wird ein nachrichtenbasierte Kommunikation verwendet. Diese Art der Kommunikation hat den Vorteil, dass die Tasks keine gemeinsamen Daten teilen müssen, welche durch einen Mutex oder Semaphor synchronisiert werden müssen. Dies führt dazu, dass sich keine Tasks gegenseitig bei Datenzugriffen blockieren und diese somit zeitlich entkoppelt werden. Die zeitliche Entkoppelung verhindert, dass die konstante Framerate aufgrund von Blockierungen nicht eingehalten werden kann.

3.5 User-Ebene

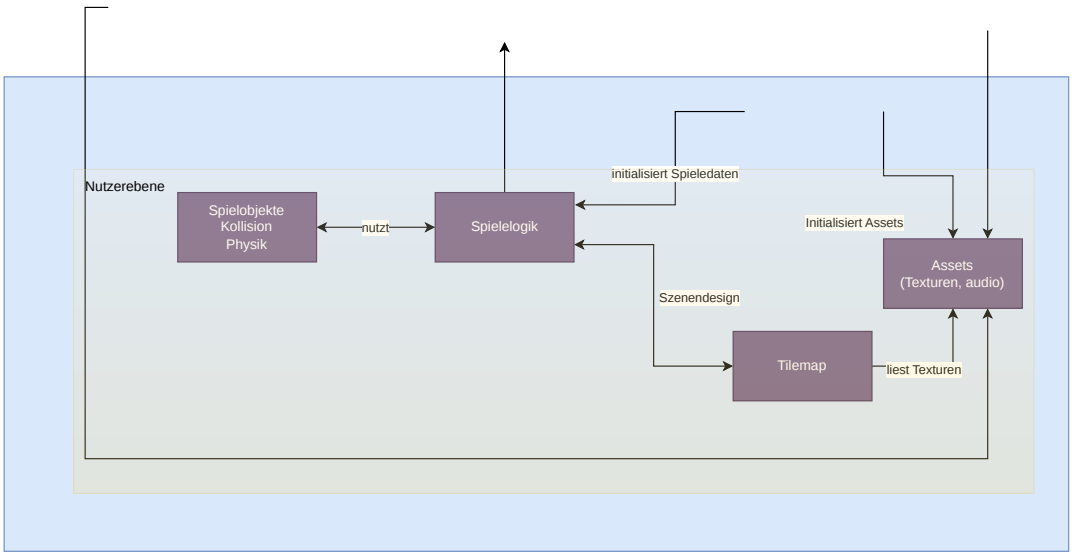


Abbildung 3.5: Architektur der User-Ebene

Die User-Ebene, zu sehen in Abbildung 3.5, setzt den Teil der Anforderungen um, bei denen der Nutzer direkt mit der Game-Engine interagieren muss, um z.B. Spieledaten speichern zu können oder eigene Spielelogik programmieren zu können.
Um dem Nutzer die Möglichkeit zu geben, Spieledaten speichern zu können, werden sogenannte Assets implementiert, wobei es sich um zwei Klassen „Texture“ und „Sound“ handelt. Diese Klassen dienen als Datenträger für Audio- und Texturdaten und können von dem Nutzer für die Nutzung im Spiel programmiert und in einem „Assetsbereich“

hinterlegt werden, damit diese global zugreifbar sind.
Damit der Nutzer das Spiel bzw. Level grafisch designen kann, soll eine TileMap zur Verfügung gestellt werden. Diese teilt den Bildschirm in gleichgroße, quadratische Bereiche ein, wobei die Größe der Teilbereiche von dem Nutzer gewählt werden kann und diesen Bereichen kann dann eine Textur zugewiesen werden. Eine weitere Möglichkeit, Levels grafisch zu designen, wäre, den Hintergrund bzw. das Level durch eine große Textur zu bestimmen. Dieser Ansatz eignet sich jedoch nicht besonders, da hierfür sehr große zusammenhängende Texturen gespeichert werden müssen, wobei bei der TileMap Texturen wiederverwendet werden können, um die Szene zu designen, da man z.B. zwei Tiles die gleiche Textur zuweisen könnte. Daher hat man dadurch die Möglichkeit, Speicherplatz zu sparen. Zudem ermöglicht die einzelne Auswahl der Texturen je Tile ein einfaches Ändern des Designs, da nur kleine Texturen ausgetauscht werden müssen. Damit der Nutzer eigene Spielelogik implementieren kann, muss dafür eine Schnittstelle vorhanden sein. Diese soll auch als Klasse umgesetzt werden, welche eine Funktion als Möglichkeit der Implementierung bereitstellt. Diese Funktion wird demnach in der Logic-Task ausgeführt, um das Spiel zu simulieren. Außerdem muss die Schnittstelle Zugriff auf die TileMap haben bzw. diese bereitstellen, damit der Nutzer diese bearbeiten kann. Damit der Nutzer Zugriff auf alle Eingabeereignisse hat und diese in der Implementierung nutzen kann, bekommt die Spielelogik-Schnittstelle von der Logic-Task Zugriff auf diese über einen Inputservice. Zudem ermöglicht die Spielelogik-Schnittstelle es dem User über einen Audioservice, Befehle an die Audio-Task zu schicken, damit Soundeffekte abgespielt werden können.
Um die Anforderung auf automatische Kollisionsüberprüfung und simple Physik-Möglichkeiten zu erfüllen, werden dem Nutzer ein „Collider“ und „Physics“ bereitgestellt, welche auch über Klassen implementiert werden sollen. Um die Nutzung von Physik, Kollision und Texturen zusammenzukapseln und zu vereinheitlichen, werden „Spieleobjekte“ als Basis zur Verfügung gestellt. Diese dienen als Basis und halten Eigenschaften wie Position, Physikeigenschaften, den „Collider“ und sind somit logische Einheiten, die in der Spielelogik-Implementierung genutzt werden können, um redundanten Datenkoppelungen zu verhindern und diese schon bereitzustellen. Diese werden auch als Klasse implementiert, da hierbei Polymorphie genutzt werden kann, um die Basis um Daten und Funktionen erweitern zu können. Außerdem ist diese Vereinheitlichung für den „Engine-Kern“ wichtig, damit dadurch automatisch der Spielzustand eingefroren werden kann und daraufhin gerendert werden kann, da man dafür mindesten alle im Spiel genutzen Texturen je „Objekt“ und deren Position benötigt. Dadurch muss der Nutzer dementsprechend auch nicht selber implementieren, wie genau der Spielzustand gekapselt werden muss.

4 Umsetzung der Hardware

Um der eingebetteten Game-Engine ihre Hardware-Grundlagen zu geben, wird in dieser Arbeit ein Handheld-ähnliches System gebaut, welches einen Microcontroller, einen Bildschirm, Eingabegeräte und Audio-Ausgänge umfasst. Im Folgenden werden die verwendeten Bauteile dargestellt, der Aufbau des Systems beschrieben und die Umsetzung protokolliert.

4.1 Verwendete Bauteile

Um den Handheld hardware-seitig umzusetzen, wurden die im Folgenden vorgestellten Bauteile genutzt.

4.1.1 Microcontroller

Als Microcontroller wird ein ESP32-P4 verwendet, da dieser mit seinem 360 MHz Prozessor [7] ausreichend Rechenleistung bietet, um komplexere Aufgaben wie Rendering oder Spielelogik auszuführen. Außerdem spricht für den ESP32-P4, dass ausreichend GPIOs zur Verfügung stehen und viele unterschiedliche Schnittstellen unterstützt werden [7], um die restlichen Bauteile des Systems zu steuern. Zusätzlich ermöglicht die Entwicklungsumgebung (ESP-IDF) eine einfache Programmierung des Microcontrollers und der Peripherie und bietet eine Integration von FreeRTOS an [6], sodass die Software-Ebene der Game-Engine auch auf dem Microcontroller umgesetzt werden kann. Außerdem bietet der ESP32-P4 mit 16 MB Flash-Speicher [4] ausreichend Speicherplatz, um mögliche Texturen oder Soundeffekte zu speichern. Durch die im ESP32-P4 eingebaute Floating-Point-Unit wird es ermöglicht, schnell mit Fließkommazahlen arbeiten zu können [7]. Dies eignet sich besonders, um Spielzustände möglichst schnell, möglichst genau berechnen zu können. Des Weiteren spricht der niedrige Stromverbrauch von 92 mA bis 123 mA bei einer Betriebsspannung von 3,3 V während einer Dual-Core Datenverarbeitung bei einer Taktrate von 360 MHz [7] gegen die Notwendigkeit eines größeren Boards wie z.B. einem Raspberry Pi.

4.1.2 Bildschirm

Als Bildschirm wird ein 2,8 Zoll großes LCD Display mit einer Auflösung von 240 x 320 Pixeln verwendet [24]. Dies ermöglicht eine angenehme Darstellung von Pixelgrafiken, ohne einen zu hohen Rechenaufwand durch eine zu hohe Auflösung zu fordern. Für die

Kommunikation zwischen Microcontroller und Display steht der ILI9488 Treiber zur Verfügung. Dieser unterstützt eine Anbindung über das I80-Interface [24]. Dieses ermöglicht einen parallelen Datentransfer, was zu geringeren Transferzeiten als bei z.B. Displays, welche über das Serial Peripheral Interface (SPI) angebunden werden, führt und somit höhrere Frameraten möglich werden. SPI ermöglicht hierbei nur eine serielle Datenübertragung [17]. Der I80-Bus benötigt wegen der parallelen Datenübertragung zwar mehr Pins als z.B. der SPI-Bus, diese werde aber in ausreichender Stückzahl vom Microcontroller zur Verfügung gestellt. Es könnten noch andere Bildschirme mit noch schnelleren Datenübertragungs-Interfaces wie z.B. dem DSI-Interface in Betracht gezogen werden. Diese sind jedoch weniger geläufig und deutlich weniger Microcontroller bieten eine Möglichkeit, diese Schnittstelle zu verwenden, was die Reproduktion dieser Arbeit erschweren würde. Außerdem unterstützt das Display das RGB565-Format [24], wobei Farben einzelner Pixel in einer 16-Bit Zahl dargestellt werden können. Dies bietet die Möglichkeit bis zu 65536 unterschiedliche Farben darzustellen, wobei der Speicherbedarf pro Pixel und die resultierenden zu übertragenden Datenmengen relativ gering bleiben. Somit wird auch kein externes Speichermedium für Spieletexturen benötigt.

4.1.3 Eingabe

Für die Eingabe werden mechanische Druckknöpfe verwendet [19]. Diese erzeugen je nach Verschaltung einen High-oder Low-Pegel beim Drücken des Knopfes und jeweils den gegenteiligen Pegel beim Loslassen. Dies eignet sich besonders für die Game-Engine, da diese den Input durch Interrupts verarbeiten soll und der Pegelwechsel Interrupts auslösen kann. Andere Eingabegeräte, wie z.B. Joysticks, eignen sich nicht für das Design, weil diese konstant Werte liefern und durch Polling ausgelesen werden müssen.

4.1.4 Audio

Für die Tonausgabe wird ein passiver Piezo-Buzzer genutzt. Dieser ermöglicht die Ausgabe unterschiedlicher Töne durch Änderung der Ausgabefrequenz. Somit können Töne und Melodien gespielt werden, ohne viel Rechenaufwand für Audio-Mixing aufwenden zu müssen. Zudem eignet sich der Piezo-Buzzer besonders als Audioausgabe, da wenig Speicher gebraucht wird, um Soundeffekte zu speichern. Somit wird auch kein externes Speichermedium notwendig.

4.2 Aufbau des Systems

Der Schaltplan (siehe Abb. 4.1) zeigt die Zusammensetzung der einzelnen Bauteile. Das gesamte System wird über eine externe Stromquelle versorgt, welche an zwei Power Pins angeschlossen wird. Zum Entwickeln bietet der ESP32-P4 auch eine Stromversorgung über

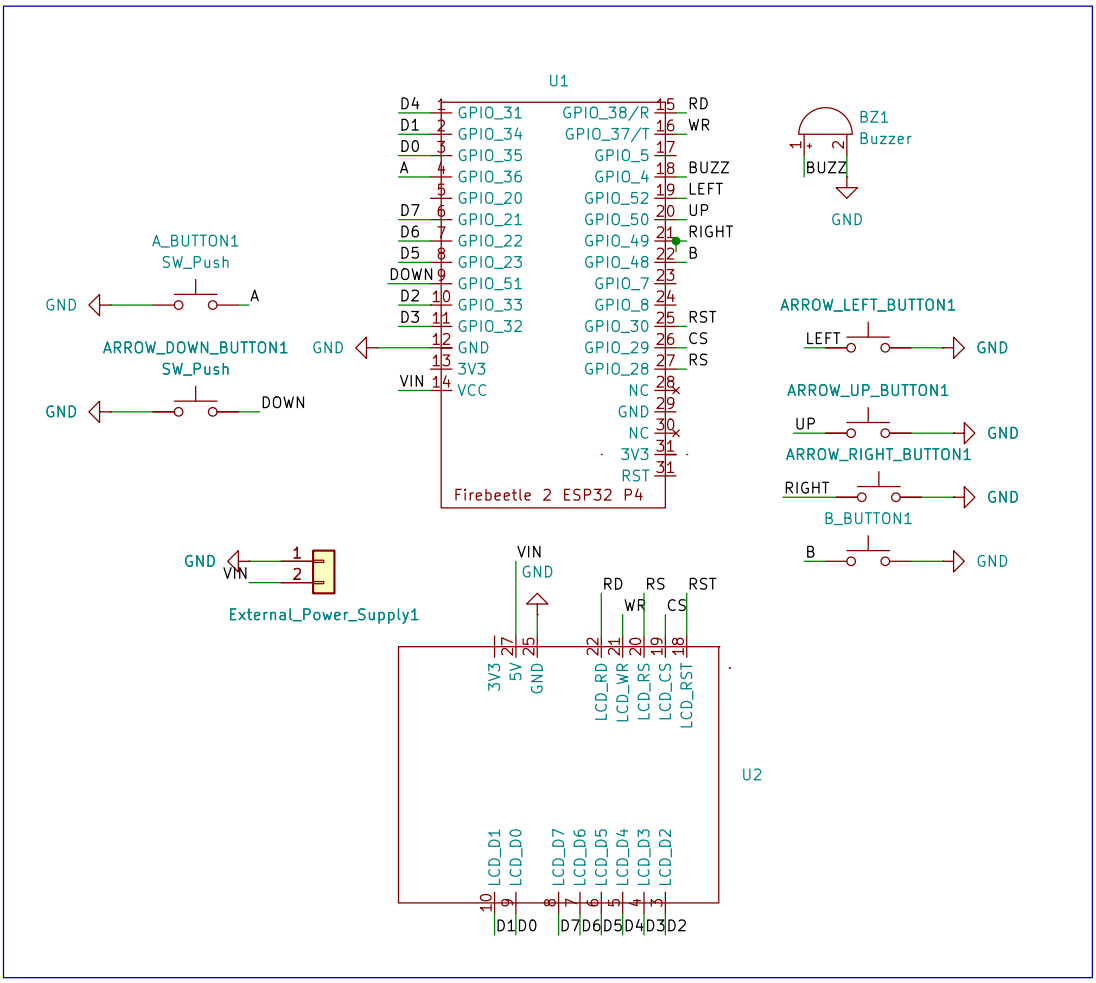


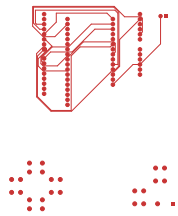
Abbildung 4.1: Schaltplan des Handhelds

USB-C [7]. Der Piezo-Buzzer wird an einen PWM-fähigen Pin angeschlossen, um den Buzzer mit unterschiedlichen Frequenzen versorgen zu können [14]. Insgesamt werden sechs mechanische Knöpfe genutzt, um ein „Steuerkreuz“ und zwei „Aktionsknöpfe“ darzustellen. Technisch stellen die Druckknöpfe zwei Pins bereit, wobei je Knopf ein Pin mit Ground und ein Pin mit einem GPIO-Pin des Microcontrollers verbunden wird (siehe Abb. 4.1). Durch diese Verschaltung liegt bei einem Knopfdruck das Signal LOW bzw. 0 am GPIO-Pin an und im nicht gedrückten Zustand HIGH bzw. 1 an. Ebenso wird der gewählte Bildschirm in das System eingebunden, indem die acht Pins der Datenleitung und die fünf Steuerpins mit GPIO-Pins des Microcontrollers verbunden werden. Die genauen Verbindungen können in Abbildung 4.1 anhand der Labels an jedem Pin abgelesen werden.

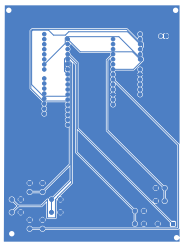
4.3 Praktische Umsetzung

Zu Beginn der Entwicklung und Implementierung der Game-Engine wurde die entworfene Schaltung zuerst auf einem Steckbrett umgesetzt, um das System testbar und noch anpassbar zu halten. Umgesetzt wird das Design, indem alle Bauteile auf einem selbstentworfenen Printed Circuit Board (PCB) gelötet werden. Betrieben wird die Schaltung über Batterien, wobei ein Spannungswandler die Eingangsspannung der Schaltung auf 5 V anpasst.

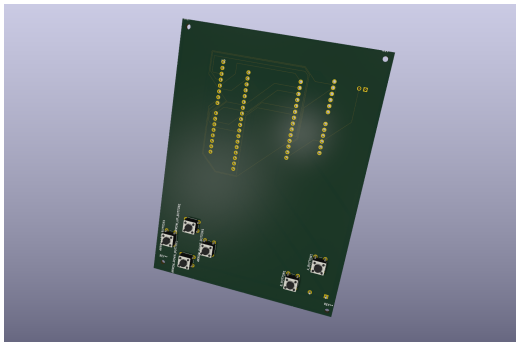
Das Erstellen der PCB wurde mit Hilfe der Software KiCad getätigt. Die PCB (siehe Abb. 4.2) wurde so designt, dass die Druckknöpfe auf der linken Seite als „Steuerkreuz“ und auf der rechten Seite als „Aktionsknöpfe“ zugeordnet wurden. Die Maße der PCB sind 90 mm mal 120 mm. Um die Dimension möglichst klein zu halten, wird der Bildschirm auf der Vorderseite und der Microcontroller auf der Rückseite angebracht.
Die Druckknöpfe und der Piezo-Buzzer werden direkt an die Platine gelötet, der Bildschirm und der Microcontroller werden angebracht, indem Buchsenleisten auf die Platine gelötet werden und die Bauteile darauf gesteckt werden. Für die Stromversorgung wird ebenfalls eine Buchsenleiste auf die Platine gelötet. Abbildung 4.2 stellt die PCB in unterschiedlichen Darstellungen dar. 4.2(a) zeigt die PCB von vorne, 4.2(b) die PCB von hinten. Hier ist zu sehen, wie genau die einzelnen Bauelemente auf der PCB angeordnet sind. 4.2(c) und 4.2(d) zeigen die PCB in dreidimensionaler Ansicht von vorne und hinten. Hierbei sind teils schon bestückte Bauteile auf der PCB zu erkennen.



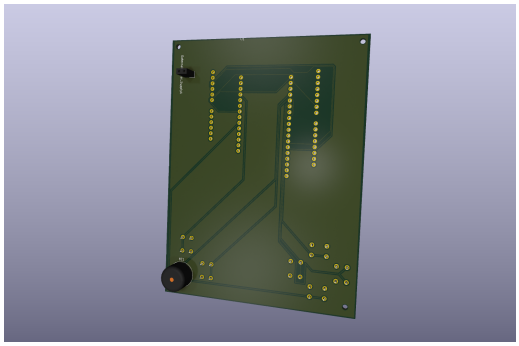
(a) Vorderansicht 2D



(b) Rückansicht 2D



(c) Vorderansicht 3D



(d) Rückansicht 3D

Abbildung 4.2: Layout und 3D-Ansicht der PCB

5 Umsetzung der Software

Die Software-Ebene der Game-Engine wird mit Hilfe des Echtzeitbetriebssystems FreeRTOS umgesetzt, da dieses direkt in der Entwicklungsumgebung des ESP32-P4 „mitintegriert“ ist. Im Folgenden wird die Umsetzung der Software-Schicht beschrieben und erläutert. Hierbei wird die Umsetzung des „Engine-Kerns“ und der „Nutzer-Ebene“ zusammen erläutert, da diese sehr ineinander greifen.

5.1 Eingabe

Im Folgendem wird die Umsetzung der Eingabe erläutert.

5.1.1 Anbindung der physischen Knöpfe

Für die Umsetzung der Eingabe müssen die Druckknöpfe zu Beginn über die ESP-IDF Schnittstelle für GPIO-Pins angeschlossen werden [10]. Hierfür werden die einzelnen GPIO-Pins, an denen die Druckknöpfe angeschlossen sind, über die Struktur `gpio_config_t` [10] konfiguriert. Hierbei werden Daten wie der Eingangspin oder der Modus (Eingabe / Ausgabe) konfiguriert. Besonders wichtig sind die Einstellungen `.intr_type` und `.pull_up_en`. Durch das Setzen der Einstellung `.intr_type` auf `GPIO_INTR_ANYEDGE` wird aktiviert, dass die Druckknöpfe bei einer steigenden oder sinkenden Flanke einen Interrupt auslösen [10], welcher daraufhin durch eine ISR behandelt werden kann. Somit ist diese Einstellung essentiell, um die Eingaben in der Game-Engine durch Interrupts abzuarbeiten. Die Einstellung `.pull_up_en` ist wichtig, da diese einen softwareseitigen Pull-Up-Widerstand einschaltet [10]. Bei diesem Widerstand handelt es sich um einen Widerstand, der den GPIO-Pin auf ein HIGH Signal hochzieht [2]. Dies wird benötigt, da am GPIO-Pin nur ein Signal anliegt, wenn der Knopf gedrückt wird. Wird der Knopf nicht gedrückt, handelt es sich bei dem Pin ohne Pull-Up Widerrstand um einen „Floating Input“. „Floating Input“ bedeutet, dass in diesem Fall kein festes Signal an dem Pin definiert ist und somit fallende oder steigende Flanken ausgelöst werden können, ohne dass der Knopf gedrückt wird [12]. Daraus würde folgen, dass ohne Pull-Up-Widerstand Eingaben entstehen können, ohne dass diese tatsächlich getätigt wurden. Abschließend wird die Konfiguration über die Funktion `gpio_config(const gpio_config_t *pGPIOConfig)` [10] übernommen.

5.1.2 ISR

Damit die Hardware-Interrupts verarbeitet werden und für den späteren Gebrauch in der Game-Engine zur Verfügung gestellt werden können, muss eine ISR zur Verfügung stehen, die diese abarbeitet. Es empfiehlt sich, eine ISR nach dem „Deferred Interrupt Processing“ Design zu entwickeln. Dieses sagt aus, eine ISR möglichst kurz zu halten, da eine ISR immer blockiert und somit bei zu langen Blockierzeiten Jitter zu der Ausführungszeit von einzelnen Tasks hinzufügen, und diese in einer Task zu verarbeiten [1]. Der Jitter wäre für das Design sehr ungünstig, da somit eine stabile Framerate gefährdet ist. Daher können z.B. nicht direkt Spielefunktionen an den Interrupt angebracht werden, da diese teils zu aufwendig sein können. Dementsprechend wird die Interrupt-Verarbeitung wie folgt umgesetzt:

```
1: Function InputHandler(void *button_type)
2: aktuellesSignal ← aktuell anliegendes Signal
3: if timeNow - lastTime[button_type] < threshold then
4:     return
5: end if
6: if lastState[button_type] == aktuellesSignal then
7:     return
8: end if
9: lastState[button_type] ← aktuellesSignal
10: lastTime[button_type] ← timeNow
11: if aktuellesSignal == 0 then
12:     inputType ← PRESSED
13: else
14:     inputType ← RELEASED
15: end if
16: event ← inputType, button_Type, timeNow
17: verschicke event
```

Algorithmus 5.1: ISR für Hardware-Interrupt Verarbeitung

Triggert das Betätigen eines Druckknopfes eine Änderung des anliegenden Signals, wird ein Hardware-Interrupt ausgelöst, welcher durch eine ISR behandelt wird. Diese ISR erzeugt ein `Inputevent`. Bei dem `Inputevent` handelt es sich um eine Struktur, welche die Art des Inputs (gedrückt oder losgelassen), den genauen Druckknopf und den Zeitpunkt des Aufkommens speichert. Das genaue Inputevent wird wie im Algorithmus 5.1 bestimmt. Zuerst wird das aktuell anliegende Signal ausgelesen. Anhanddessen kann man bestimmen, ob es sich um eine sinkende oder steigende Flanke gehandelt hat, die den Interrupt ausgelöst hat, da nach dem Wechsel des anliegenden Signals, das Signal des Endzustandes an dem GPIO-Pin anliegt. Aufgrund der Verschaltung (siehe 4), liegt am Pin LOW bzw. 0 an, wenn der Druckknopf gedrückt wird, und HIGH bzw. 1, wenn der Druckknopf nicht

gedrückt wird. Liegt also zum Zeitpunkt der ISR 0 an, handelt es sich am einen Druck auf den Knopf, liegt 1 an, wurde der Knopf losgelassen. Dargestellt wird die Art des Inputs anhand der Enumeration `INPUT_TYPE`. Der Knopf, der den Interrupt ausgelöst hat, wird über die Funktionssignatur übergeben und direkt im Inputevent gespeichert. Um mechanisches Prellen zu verhindern, wird ein Sicherheitsmechanismus in die ISR eingebaut. Bei mechanischem Prellen handelt es sich um ein physikalisches Phänomen, wobei sich das neu anliegende Signal beim Schließen eines Schaltkreises erst einpendelt, bevor es stabil am Pin anliegt [20]. Das heißt, dass z.B. ein einzelner Knopfdruck mehrere steigende und sinkende Flanken auslösen kann. Um dies zu verhindern, verarbeitet der ISR nur neue Interrupts an jedem Knopfpin, wenn diese eine bestimmte Zeitspanne bis zum letzten akzeptierten Interrupt überschreiten. Bei dem Entwickeln und Testen der Eingabe, ist aufgefallen, dass auch ungültige Zustandswechsel aufgekommen sind. Beispielsweise wurde ein Drücken eines Knopfes registriert, obwohl dieser losgelassen wurde. Um dies zu verhindern wird neben der letzten Zeit auch der letzte stabile Zustand an dem Knopfpin gespeichert. Der ISR validiert dann, ob es sich um einen gültigen Zustandswechsel handelt, sodass nur abwechselnd ein Drücken und Loslassen akzeptiert wird. Wird der Interrupt akzeptiert, wird das erzeugte `InputEvent` an die Logic-Task zum Verarbeiten verschickt. Zum Verschicken der `InputEvents` wird eine Queue [23] genutzt. Die Queue eignet sich besonders für die Kommunikation zwischen Interrupt und Logic-Task, da sie zum einen die nachrichtenbasierte Kommunikation ermöglicht, da die Eingabeevents als Nachrichten in die Queue hineingelegt werden können. Daher werden allgemein in der ganzen Game-Engine Queues als Kommunikationsmittel zwischen Tasks genutzt. Zum anderen ermöglichen Queues, dass Nachrichten bzw. Daten sicher innerhalb von ISRs in diese gelegt werden können, da das Übertragen nicht blockiert [23].
Diese ISR wird anschließend über die Funktion `gpio_isr_handler_add(gpio_num_t gpio_num, gpio_isr_t isr_handler, void *args)` [10] an jeden Pin, mit dem ein Druckknopf verbunden ist, angebracht. Als Paramter wird dem ISR ein dem Druckknopf entsprechender Wert der Enumeration `BUTTON_TYPE` übergeben, um zwischen den einzelnen Knöpfen unterscheiden zu können. Um je Pin einen ISR registrieren zu können und nicht einen globalen ISR regstrieren zu müssen, wird außerdem die Funktion `gpio_install_isr_service(int intr_alloc_flags)` aufgerufen [10].

5.1.3 Input-Service

Damit die `InputEvents` von dem Nutzer bei dem Implementieren eigener Spiele genutzt werden können, wird diesem ein Input-Service bereitgestellt. Dieser Input-Service wird als Klasse `InputService` umgesetzt. Zu den Aufgaben des `InputService` gehört es zum einen aus den rohen Eingabeevents für den Nutzer nutzbare Eingabe zu verarbeiten und zum anderen, die aktuellen Zustände und Daten der einzelnen Knöpfe zu verwalten und bereitzu-

stellen. Zum verwalten der Daten hält der `InputService` sechs Objekte der Klasse `Button`. Da sowohl auf steigende, als auch auf fallende Flanken reagiert wird, ermöglicht dies drei Zustände, die ein Druckknopf haben kann. Ein Druckknopf kann gedrückt worden sein, gehalten werden oder losgelassen worden sein. Dementsprechend speichert die Klasse `Button` diese drei Zustände des jeweiligen Knopfes und außerdem die letzte Zeit, als der Knopf gedrückt und losgelassen wurde. Aus diesen Daten kann der `InputService` dem Nutzer die Abfragen `wasButtonReleased(BUTTON_TYPE button)`, `wasButtonPressed(BUTTON_TYPE button)` und `isButtonHeld(BUTTON_TYPE button)` bereitstellen, um in der Spielelogik auf die Eingaben reagieren zu können. Ein Knopf gilt hierbei ab dem Moment als gedrückt gehalten, sobald dieser einmal gedrückt wurde. Der Zustand gilt so lange, bis der Knopf wieder losgelassen wird. Durch die gespeicherten Zeiten lässt sich ebenfalls überprüfen, ob ein bestimmter Knopf für eine gewisse Zeit gedrückt wurde. Diese Abfrage wird durch `wasButtonHeldFor(BUTTON_TYPE button, uint64_t time)` bereitgestellt. Verarbeitet werden die rohen InputEvents durch die Funktion `handleInputEvent(InputEvent event)`. Die einzelnen Druckknöpfe werden wie im ISR durch die Enumeration `BUTTON_TYPE` unterschieden. Zum Beispiel zu zählen, wie oft ein Knopf gedrückt wurde, wird bewusst verzichtet. Begründet wird dies in Kapitel 5.2, wenn die Art der Verarbeitung der Inputevents beschrieben wird.

5.2 Logic-Task

Um die Spielelogik regelmäßig zu aktualisieren, wird eine Logic-Task implementiert. Hierfür wird die Task-API von FreeRTOS [23] genutzt, da diese die Möglichkeit bietet, periodische Tasks zu implementieren. Die Logic-Task wird wie in der Abbildung 5.1 umgesetzt. Bevor die Spielelogik aktualisiert wird, werden zuerst alle vom ISR (siehe Kap. 5.1) einkommenden Inputevents verarbeitet, indem diese dem `InputService` zur Verarbeitung übergeben werden. Dafür wird mit `uxQueueMessagesWaiting()` [23] bestimmt, wie viele Inputevents sich in der `inputQueue` befinden, woraufhin diese Anzahl mit `xQueueReceive()` [23] ausgelesen wird. Die Inputevents werden zuerst verarbeitet, damit die Spielelogik auf diese reagieren kann. Hierbei wird bewusst nicht mitgezählt, wie oft z.B. ein Knopf gedrückt wurde. Grund dafür ist, dass die Logic-Task wie in Kapitel 5.4 einmal je Frame ausgeführt

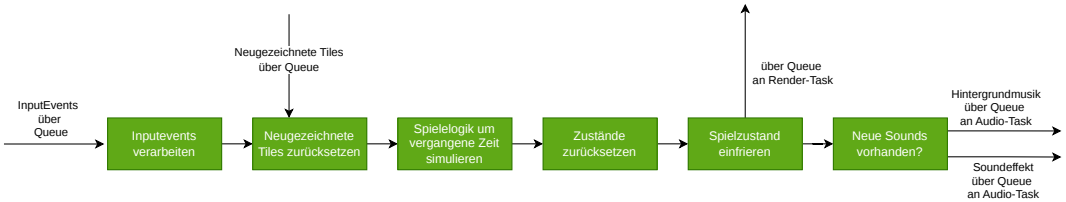


Abbildung 5.1: Ablauf der Logic-Task

wird. Bei einer Framerate von beispielsweise 30 FPS, also einer Framezeit von ca. 33.3 ms würden die `InputEvents` also alle 33.3 ms abgearbeitet werden. In so einer Zeitspanne würde man es nicht schaffen, mehrmals auf einen Knopf zu drücken. Dementsprechend sollte also nach dem umgesetzten Design auch keine Eingabe verpasst werden. Daraufhin werden alle gezeichneten Bereiche des Bildschirmes zurückgesetzt. Dies ist notwendig, da die Render-Task nur Bereiche des Bildes neu zeichnet, die sich verändert haben. Genaueres dazu wird im Kapitel 5.3 erläutert. Damit die Spielelogik unabhängig von der Framezeit simuliert werden kann und z.B. Bewegungen unabhängig von der Framerate immer gleichmäßig verlaufen, wird außerdem die vergangene Zeit zwischen dem Start des letzten und des aktuellen Jobs gemessen und der Spielelogik beim Aktualisieren übergeben. Nachdem die Spielelogik aktualisiert wurde, müssen einige Eingabezustände über die `InputService`-Funktion `resetInputAfterFrame()` zurückgesetzt werden. Dies hat den Grund, dass die Eingaben „Knopf X wurde gedrückt“ oder „Knopf X wurde losgelassen“ nur einen Job der Task lang gültig sind und daraufhin zurückgesetzt werden müssen, damit z.B. auf einen Knopfdruck nicht mehrmals reagiert wird.
Außerdem wird nach dem Aktualisieren der Spielelogik der aktuelle Spielzustand „eingefroren“, damit die Render-Task diesen auf dem Bildschirm zeichnen kann. Gespeichert wird der Spielzustand durch die Struktur `WorldSnapshot`. Der genau Inhalt wird in Kapitel 5.3 und die genaue Erzeugung in Kapitel 5.7 erläutert. Nach Einfrieren des Spielzustandes wird dieser über die Queue `renderQueue` verschickt. Hierbei handelt es sich um eine Queue mit genug Platz für genau einen Spielzustand. Diese Größe der Queue ermöglicht in Kombination mit der Funktion `xQueueOverwrite()` [23], welche für Queues der Länge 1 konzipiert ist und den Inhalt dieser immer überschreibt, dass der Render-Task immer der aktuelleste Spielzustand zum Rendern zur Verfügung steht. Im Vergleich zu einer größeren Queue können sich dadurch keine Spielzustände stauen, sollte die Render-Task nicht mit dem Takt hinterherkommen, und somit wird kein Zustand aus der Vergangenheit gezeichnet, obwohl der wirkliche Spielzustand schon weiter fortgeschritten ist. Wird in der Spielelogik der Befehl ausgelöst, einen Soundeffekt oder Hintergrundmusik zu starten, werden diese Befehle über zwei Queues an die Audio-Task verschickt. Nach Verschicken der Befehle wird der `AudioService`, welcher die Verwaltung des Audios bereitstellt, zurückgesetzt, damit die gleichen Audio-Befehle nicht mehrfach verschickt werden. Damit die Task periodisch ausgeführt wird, wird zum Schluss die Funktion `vTaskDelayUntil()` [23] der Task-API aufgerufen. Diese Funktion blockiert die Ausführung der Logic-Task relativ zum letzten Startzeitpunkt des Jobs für die übergebene Zeitspanne. Als Zeitspanne wird hier die Framerate mit der Formel 1000[ms]/FPS in die Periodenzeit umgerechnet und diese wird mit Hilfe der Funktion `pd_MS_To_Tick()` in die von der Funktion akzeptierten Größe umgerechnet. Dadurch wird die Spielelogik gleichmäßig einmal pro Frame ausgeführt, damit die stabile Framerate erreicht wird (siehe Kap. 5.4).

5.3 Render-Task und Grafik

Damit das Spiel grafisch dargestellt werden kann und der Nutzer dieses auch grafisch gestalten kann, muss die Game-Engine mehrere Komponenten bereitstellen. Diese werden im Folgenden erläutert.

5.3.1 Render-Task

Function `renderTask(void *param)`
Initialisiere Zeitvariablen
while true **do**
  lese den aktuellsten Snapshot
  for tile im Snapshot **do**
    nimm einen freien Buffer
    markiere den Buffer als genutzt
    kopiere Textur der Tile in den Buffer
    for object im Snapshot **do**
      if object in oder berührt die tile **then**
        kopiere Textur des objects in den Buffer
      end if
    end for
    schicke den Buffer an den Bildschirm an die Position der tile
    Warte bis zum nächsten Ausführungszeitpunkt
  end for
end while

Algorithmus 5.2: Ablauf der Render-Task

```
1    bool onTransfer(esp_lcd_panel_io_handle_t panel_io,
        esp_lcd_panel_io_event_data_t *edata, void *user_ctx){
2    bool tmpForTok = false;
3    RedrawedTile t = {};
4    xQueueReceiveFromISR(transferTokens, &tmpForTok, 0);
5
6    uint16_t* buf;
7
8    if(xQueueReceiveFromISR(bufferInUseQueue, &buf, 0) == pdTRUE){
9        xQueueSendFromISR(freeBufferQueue, &buf, 0);
10   }
11   if(xQueueReceiveFromISR(tileInTransferQueue, &t, 0) == pdTRUE){
12       xQueueSendFromISR(redrawedTilesQueue, &t, 0);
13   }
14   return false;
15 }
```

Listing 5.1: Callback nach einem Transfer

Die Render-Task dient dafür, das Bild aus dem aktuellen Spielzustand zu generieren. Es gibt unterschiedliche Ansätze, das Bild zu generieren. Beispielsweise kann in jedem Job der Render-Task das gesamte Bild generiert und an den Bildschirm verschickt werden oder es werden nur Teilbereiche generiert, welche neu gezeichnet werden müssen. Da die Datenübertragungsrate des Bildschirmes begrenzt ist und somit ein Flaschenhals für die Framerate sein kann, wird in dieser Arbeit ein Rendering-Algorithmus nach dem „Dirty Rectangle“ Prinzip verfolgt. Hierbei werden nur Teilbereiche des Bildes neugeneriert, wo es zu Änderungen im Spielzustand gekommen ist [8]. Dadurch wird die Datenmenge, die an den Bildschirm versendet werden muss, möglichst minimiert und eine konstante und möglichst hohe Framerate kann sichergestellt werden. Durch das „Dirty Rectangle“ Prinzip kommt zwar mehr Verwaltungsaufwand hinzu, dieser bleibt jedoch relativ gering, da sich die Verwaltung in die im Design erläuterte TileMap integrieren lässt. Um die Render-Task praktisch umzusetzen, wird der Algorithmus 5.2 in einer Task umgesetzt: Zu Beginn jedes Render-Task-Jobs wird der aktuellste, zur Verfügung stehende „Snapshot“ gelesen, welcher von der Logic-Task generiert wurde. Dazu wird mit Hilfe der Funktion `xQueueReceive()` der Wert aus der `renderQueue` gelesen.
Der gelesene Snapshot enthält alle Tiles der TileMap (siehe Kap. 5.3.2) und alle Spielobjekte (siehe Kap. 5.8) im Spiel. Die Render-Task iteriert nun über jede `Tile`. Muss die `Tile` neu gezeichnet werden, wird ein freier Buffer genommen und die Textur der Tile wird dort hineinkopiert. Bei dem Buffer handelt es sich um einen Array in der selben Größe wie eine `Tile`. In diesem wird der gesamte Bildausschnitt, welcher in jeweiligen `Tile` enthalten ist, erzeugt. Gibt es keinen freien Buffer, wird die in der Iteration aktuelle `Tile` übersprungen und im nächsten Job gerendert. Um die freien Buffer zu verwalten, wird eine Queue bereitgestellt, die die Adressen der Bufferarrays speichert. Mit `xQueueReceive()` wird dementsprechend ein freier Buffer aus der `freeBufferQueue` gelesen. Der Buffer, der zum Rendern der Tile benutzt wird, wird auch direkt als „genutzt“ markiert. Hierfür wird die Adresse des genutzten Buffers in die `bufferInUseQueue` gelegt. Dies ist notwendig, damit nicht ein Buffer beschrieben wird, der aktuell an den Bildschirm gesendet wird, parallel neu beschrieben wird. Die Queue schützt somit einen Buffer vor Race Conditions, sodass kein Bild zwei unterschiedliche Zustände gleichzeitig darstellt. Insgesamt werden zehn Buffer zur Verfügung gestellt.
Nachdem die Textur der Tile in den Buffer kopiert wurde, wird über jedes Spielobjekt iteriert. Befindet sich ein Objekt innerhalb der `Tile`, die aktuelle gerendert wird, wird dessen Textur in den Buffer kopiert. Nachdem alle Objekte überprüft wurden, wird der generierte Buffer über `esp_lcd_panel_draw_bitmap()` [13] an den Bildschirm verschickt.
Um der Logic-Task zu signalisieren, dass eine Tile neu gezeichnet wurde und der Zustand dieser somit auf „gezeichnet“ zurückgesetzt werden kann, wird eine `Tile` als „im Zeichnen“ vermerkt, indem diese in die `tileInTransferQueue` gelegt wird. Diese Queue wird später genutzt, um die `Tile` komplett freizugeben.

Die Logic-Task muss über die gezeichneten Tiles informiert werden, damit deren Zustand zurückgesetzt werden kann und diese dann nicht mehr vom Rendering Algorithmus neu gezeichnet werden, obwohl diese dies nicht benötigen. Prinzipiell könnte die Logic-Task alle Tiles selber zurücksetzen, dies könnte jedoch zu Fehlern im Gesamtbild führen, falls eine Tile nicht vom Rendering gerendert wurde, da z.B. kein freier Buffer zur Verfügung stand. Würde die Logic-Task selber alle Tiles zurücksetzen, würde somit die Information verloren gehen, dass die Tile noch neu gezeichnet werden muss. Die LCD-API des ESP32 bietet es an, eine Callback-Funktion zu registrieren, welche ausgeführt wird, wenn eine Übertragung zum Bildschirm beendet wurde. Die Registrierung erfolgt über die Funktion `esp_lcd_panel_io_register_event_callbacks()` [13]. So eine Callback-Funktion wird implementiert, um einen Buffer nach Fertigstellung des Transfers zum Bildschirm wieder freizugeben. Hierfür wird die Adresse des Buffers wieder in die `freeBuffer`-Queue gelegt(siehe 5.1). Außerdem wird hier über die `redrawedTile` Queue der Logic-Task übermittelt, dass eine Tile neu gezeichnet wurde. Damit die Task periodisch ausgeführt wird, wird zum Schluss die Funktion `vTaskDelayUntil()` [23] der Task-API aufgerufen.

5.3.2 TileMap

Wie in Kapitel 3 beschrieben, soll eine TileMap implementiert werden, welche dem Nutzer ermöglicht, die Spielszene bzw. den Hintergrund zu designen. Hierfür wird die Klasse `TileMap` implementiert. Diese teilt den Bildschirm in ein Raster aus gleich großen Bereichen auf, welche über die `TileMap` verwaltet werden können. Um die Daten für jeden einzelnen Teilbereich zu bündeln, wird die Klasse `Tile` angelegt. Diese hält Daten wie die Textur, die Größe des Teilbereichs und die Position im Raster. Durch diese Informationen lässt sich die `Tile` grafisch darstellen und durch die Größe und die Position im Raster lässt sich die absolute Position berechnen, um zu überprüfen, ob sich etwas in diesem Teilbereich verändert. Um zu speichern, dass eine `Tile` neu gezeichnet werden muss, wird dafür eine Flag zur Verfügung gestellt. Die `TileMap` ermöglicht es zudem, jeder einzelnen `Tile` eine Textur zuzuweisen und sie ermöglicht durch ein Überprüfen von Positionen das Triggern der Redraw-Flag, um zu signalisieren, dass eine `Tile` neugezeichnet werden muss. Da man mit der `TileMap` den Hintergrund des Spiels designt, wird diese als unterste Ebene im Rendering gezeichnet.

5.3.3 Texturen

Um Texturen verwalten zu können, wird die Klasse `Texture` zur Verfügung gestellt. Diese hält die Breite, die Höhe und die Pixeldaten der Textur. Die Pixeldaten werden als row-major-Array aus 16-Bit-Integern dargestellt, um jeden einzelnen Pixel einer Textur mit einer Zahl darstellen zu können. Außerdem hält die Klasse `Texture` eine Bitmaske für die Textur. Diese Bitmaske ermöglicht es, dass nur ausgewählte Pixel der Textur geren-

dert werden. Dies ermöglicht beispielsweise, dass ein Kreis ohne farbigen, eckigen Rand gerendert wird. Bereitgestellt wird die Bitmaske auch als row-major-Array, welches 16-Bit Werte speichert. Möchte man einen Pixel darstellen, nutzt man den Wert 0xFFFF, wenn nicht dann 0x0000. Andere Werte führen zu undefiniertem Verhalten. Gibt man keine Bitmaske an, werden alle Pixel im Rendering betrachtet. Übergibt man der `Texture`-Klasse kein gültiges Array an Pixeldaten, wird die Textur nicht gezeichnet.Übergibt man eine falsche Breite und Höhe, führt es auch zu undefiniertem Verhalten.

5.4 Timing

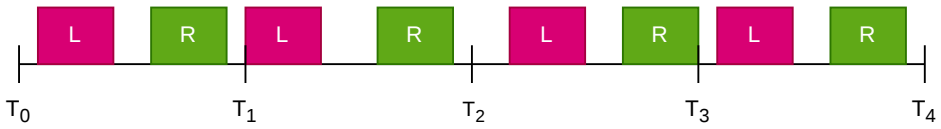


Abbildung 5.2: optimales Timing

Je Frame muss während des Verlauf eines Spieles die Logik aktualisiert und das aktuelleste Bild gerendert werden, wie in Abbildung 5.2 dargestellt. Um eine konstante Framerate und somit ein flüssiges Spieleerlebnis zu erreichen, müssen somit Logic-Task und Render-Task einmal pro Frame ausgeführt werden. Um dies zu erreichen, wird aus der angestrebten Framerate die Periodenzeit, die pro Frame zur Verfügung steht, berechnet und beide Tasks werden mit dieser Periodendauer ausgeführt. Die angestrebte Framerate lässt sich vom Nutzer über eine Config-Datei einstellen. Die Periodenzeit wird durch die Beziehung T = 1000/FPS berechnet. Wichtig ist hierbei, dass die Periodenzeit in Millisekunden angegeben wird. Die FreeRTOS-Funktion `vTaskDelayUntil()` [23] bekommt die Periodenzeit als Tick-Anzahl übergeben. Zum Umrechnen wird die Funktion `pdMS_TO_TICKS()` genutzt.
Der FreeRTOS-Scheduler führt immer die Task im Zustand ready aus, die die höchste Priorität hat [1]. Um während eines Periodendurchlaufs den Ablauf so zu takten, dass die Logic-Task vor der Render-Task ausgeführt wird, bekommt die Logic-Task eine höhere Priorität als die Render-Task. Sollte es mal zu Ungleichmäßigkeiten bei der Ausführung der Tasks kommen, wenn z.B. die Periodenzeit nicht eingehalten werden kann, sorgt die höhere Priorität der Logic-Task dafür, dass die Spielelogik möglichst aktuell bleibt. Somit bleibt der Spielverlauf gleichmäßig und nur das Bild würde eher ruckeln. In der Umsetzung der Engine läuft die Logic-Task mit der Priorität 4, die Render-Task mit der Priorität 3.

5.5 Audio

Um die Anforderungen für Audiounterstützung der Game-Engine umzusetzen, werden unterschiedliche Komponenten in den Schichten der Game-Engine umgesetzt.

5.5.1 Töne und Soundeffekte

Damit man Soundeffekte speichern und verwalten kann, wird die Struktur `Tone` und die Klasse `Sound` zur Verfügung gestellt. Bei der Struktur `Tone` handelt es sich um eine Basisstruktur, welche die Höhe eines Tons und dessen Dauer speichert. Ein Ton wird in genau dieser Form gespeichert, da zum Abspielen von Audio ein Piezo-Buzzer (siehe 4.1) genutzt wird. Dieser kann nur Töne anhand einer Frequenz erzeugen. Die Klasse `Sound` speichert primär einen Array aus Objekten der Struktur `Tone`. Dadurch wird eine Tonleiter gespeichert, welche einen Soundeffekt darstellt. Neben dem Soundeffekt selber, verwaltet die Klasse `Sound` das Abspielen des Soundeffekts, indem sie die aktuelle Zeit innerhalb des Soundeffekts speichert, diese Zeit aktualisiert und den aktuellen Ton zurückgibt. Genutzt wird dies von der Audio-Task im Game-Engine-Kern, um den passenden Ton abzuspielen. Außerdem kann man über die Klasse `Sound` einstellen, ob ein Sound nur einmalig durchgespielt oder in einer Schleife gespielt wird. Das Spielen in einer Schleife ermöglicht es z.B. durch kurze Tonleitern eine Hintergrundmusik zu erzeugen. Zudem speichert die Klasse `Sound` die Priorität des Soundeffektes. Diese wird genutzt, um nur den Soundeffekt mit der höchsten Priorität zu spielen, sollte es in der Spielelogik zu Situationen kommen, in denen mehrere Sounds abgespielt werden könnten.

5.5.2 Audio-Service

Damit der Nutzer beim Implementieren von Spielen das Abspielen von Soundeffekten auslösen kann, wird ein Audioservice zur Verfügung gestellt. Der Audioservice wird über die Klasse `AudioService` implementiert. Der Audioservice ermöglicht es dem Nutzer, Soundeffekte auf zwei „Tonspuren“ abspielen zu lassen. Die erste „Tonspur“ kann genutzt werden, um eine Hintergrundmusik einstellen und abspielen zu können, die zweite wird für aufkommende Soundeffekte genutzt. Über die Funktionen `setSoundeffect(Sound* newEffect)` und `setBackgroundMusic(Sound* newBackgroundMusic)` können die zu spielenden Sounds eingestellt werden. Beim Einstellen der Sounds wird immer auf die Priorität der Sounds geachtet, sodass nur der Sound mit der höchsten Priorität übernommen wird. Zudem ermöglicht es der Audioservice, die aktuell spielenden Sounds zu stoppen. Wie in Kapitel 5.2 erwähnt, werden nach jeder Spielelogikausführung die eingestellten Sounds von der Logic-Task zu der Audio-Task übermittelt. Hierfür werden diese aus dem Audioservice ausgelesen, sofern Sounds vorhanden sind, und über zwei Queues an die Audio-Task verschickt. Nach dem Verschicken werden die eingestellten Sounds im Audioservice zurückgestellt, um im nächsten Logic-Task Job das Einstellen wieder sauber zu ermöglichen.

5.5.3 Audio-Task

Damit die zu spielenden Soundeffekte sauber abgespielt werden können, wird eine periodische Audio-Task implementiert. Diese ist dafür zuständig, die zu spielenden Sounds bei Bedarf zu wechseln, die aktuellen Sounds in kleinen Zeitabständen zu aktualisieren und somit abzuspielen und den aktuellen Ton, der abgespielt werden muss, an die Audio-Ausgabe zu schicken.
Hierfür wird die Audio-Task wie in Abbildung 5.3 dargestellt implementiert. Zu Beginn wird überprüft, ob ein neuer Soundeffekt oder eine neue Hintergrundmusik gespielt werden soll. Hierfür wird für den Soundeffekt die Queue `soundEffectQueue` und für die Hintergrundmusik die Queue `backgroundMusicQueue` ausgelesen. Gibt es einen Soundeffekt, wird der aktuelle Sound nur überschrieben, wenn der neue Sound eine höhere Priorität hat. Dadurch wird erreicht, dass der Nutzer bestimmen kann, welche Sounds priorisiert abgespielt werden sollen. Bei der Hintergrundmusik wird jedoch die Priorität nicht beachtet. Grund dafür ist, dass diese seltener gewechselt wird und immer gewechselt werden kann, wenn der Nutzer dies möchte. Werden neue Sounds gesetzt, wird deren Abspielen gestartet, damit diese wieder von Anfang an spielen. Wurden keine neuen Sounds gesetzt, aber es gibt aktuelle Sounds, werden diese um die vergangene Zeit aktualisiert. Ist der Soundeffekt oder die Hintergrundmusik vorbei, werden die aktuellen Sounds auf leer gesetzt. Nach der Aktualisierung wird der Ton bestimmt, welcher an die Audioausgabe geschickt wird. Der Soundeffekt wird immer mit höherer Priorität als die Hintergrundmusik abgespielt. Das heißt gibt es einen aktuellen Soundeffekt, wird dessen aktueller Ton abgespielt. Gibt es keinen Sound, aber eine Hintergrundmusik, wird ihr aktueller Ton abgeschickt, ansonsten wird gar kein Ton abgespielt.
Um den aktuellen Ton zu spielen, wird die Frequenz des Tones an dem Pin erzeugt, an dem der Piezo-Buzzer angeschlossen ist. Hierfür werden die Funktionen `ledc_set_duty()`, `ledc_update_duty()` und `ledc_set_freq()` genutzt [14]. Um Soundeffekte möglichst genau zu aktualisieren, wird die Audio-Task losgelöst von der Framerate mit einer möglichst kleinen Periodenzeit ausgeführt. Diese lässt sich über eine Config-Datei einstellen und wird standardmäßig auf 5 ms gesetzt. Die Priorität der Audio-Task wird auf 5 gesetzt, damit diese immer direkt ausgeführt wird.

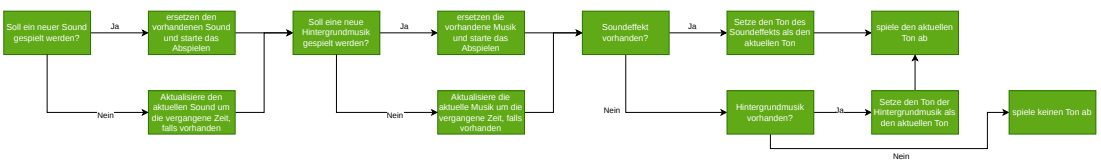


Abbildung 5.3: Ablauf der Audio-Task

5.6 Dynamische Framerateanpassung

Um die Anforderung zur dynamischen Framerateanpassung zu erfüllen, müssen einige Performancedaten während der Ausführung des Spieles gesammelt werden, um daraus eine resultierende Framerate berechnen zu können und die aktuelle Framerate dahingehend anzupassen. In dieser Arbeit werden die Ausführungszeiten der Logic-Task und Render-Task, die Deadline-Misses und Transfer-Misses, also ob das gesamte Bild vor dem nächsten Render-Task-Job am Display angekommen ist, gemessen, um darauf die anzustrebende Framerate zu bestimmen. Bestimmt wird die anzustrebende Framerate durch die Statistik-Task.

5.6.1 Ausführungszeiten

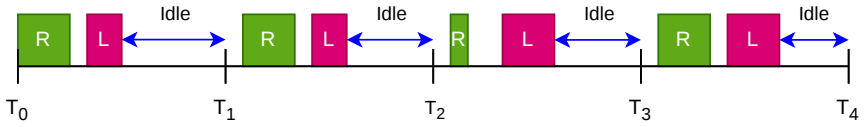


Abbildung 5.4: Periodenzeit ist zu groß

Wie in Kapitel 5.4 dargestellt, wird eine Ausführung der Logic-Task und eine Ausführung der Render-Task je Frame ausgeführt. Das heißt, dass die Ausführungszeit der Logic-Task addiert mit der Ausführungszeit der Render-Task die Periodenzeit der aktuellen Framerate ergeben sollten. Setzt man eine Framerate statisch fest, kann es sein, dass die Ausführungszeiten zusammenaddiert zu groß sind. Dies würde bedeuten, dass die statisch festgesetzte Framerate zu hoch ist. Sind die addierten Ausführungszeiten deutlich kleiner als die Periodenzeit der eingestellten Framerate, wie in Abbildung 5.4 dargestellt, kann diese noch höher gesetzt werden, da die Periodenzeit nicht komplett ausgelastet ist. Um die Auslastung der Periodenzeit bestimmen zu können, werden für Logic-Task und Render-Task die Ausführungszeiten gemessen. Hierfür wird je Task ein großer Array von 10000 Werten angelegt, der Start- Stopp-Zeit-Paare speichert. Die Zeit wird durch die Funktion `esp_cpu_get_cycle_count()` [18] gemessen. Diese liest nur den CPU Cycle Counter aus einem Register aus, was wenig Overhead mit sich bringt.

5.6.2 Deadline-Misses

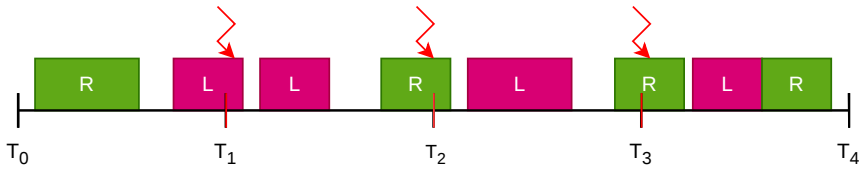


Abbildung 5.5: Deadline-Misses

Deadline-Misses treten auf, wenn eine periodische Task nach ihrer Deadline fertig wird, wie in Abbildung 5.5 dargestellt. Im Zusammenhang mit der Game-Engine bedeutet ein Deadline-Miss, dass ein Logic-Task oder Render-Task Job erst in einem späteren Frame fertig wird. Treten Deadline-Misses auf, bedeutet dies, dass die Periodenzeit der eingestellten Framerate praktisch nicht von dem System eingehalten werden kann. Um darauf reagieren zu können, wird jeder Deadline-Miss der Logic-Task und Render-Task gemessen. Hierfür wird die von der freeRTOS-API bereitgestellte Funktion `xTaskDelayUntil()` anstelle der Funktion `vTaskDelayUntil()` für das periodische Warten genutzt. Die Funktion `xTaskDelayUntil()` bietet den Vorteil, dass sie `pdFALSE` zurückgibt, wenn die nächste Startzeit der Periode einer Task in der Vergangenheit liegt [23]. Dies bedeutet dementsprechend, dass die Task ihre Deadline verpasst hat. Kommt es bei der Logic-Task oder Render-Task zu einem Deadline-Miss, signalisieren sie dies der Statistik-Task über eine Queue. Hierbei wird bei Aufkommen eines Misses ein Token in die `Deadline-MissedSignal` Queue gelegt. Bei den Tokens handelt es sich um boolsche Werte ohne Bedeutung. Diese wurden gewählt, da sie wenig speicher verbrauchen. Mit der Anzahl an Tokens in der Queue kann die Statistik-Task daraufhin die Anzahl an Deadline-Misses bestimmen.

5.6.3 Transfer-Misses

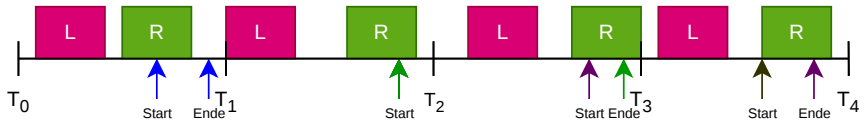


Abbildung 5.6: Transfer-Misses

Zu einem Transfer-Miss kommt es, wenn ein Transfer von Bilddaten eines vorherigen Frames noch während des nächsten Jobs der Render-Task stattfindet. Dieser Fall wird in Abbildung 5.6 dargestellt. Dementsprechend sind Transfer-Misses Signale dafür, dass die angestrebte Framerate nicht von der Bandbreite des Bildschirmes unterstützt wird und das Bild praktisch je Periode nicht schnell genug übertragen werden kann. Um Transfer-Misses messen zu können, werden die Render-Task (siehe 5.2) und die Callback-Funktion

(siehe 5.1) erweitert. Je Transferstart wird, ähnlich wie bei den Deadline-Misses, ein Token in eine Queue gelegt. Bei den Tokens handelt es sich wieder um boolsche Werte ohne Bedeutung. Die in der `transferTokens` liegenden Tokens werden dann durch den Callback (siehe 5.1) wieder freigeben, da dieser aufgerufen wird, sobald ein Transfer beendet wurde. Hinzu kommt, dass die Render-Task zu Beginn ihres Jobs überprüft, ob noch Tokens in der `transferTokens` Queue enthalten sind. Wenn ja, hat man den Fall, dass noch Daten des vorherigen Frames übertragen werden, obwohl der nächste gerendert werden muss. Kommt es zu diesem Fall, überträgt die Render-Task dies über eine Queue mit nur einem Element an die Statistik-Task als ein Signal. Der Wert, der übertragen wird, hat hierbei keine Bedeutung. Nur durch den Inhalt der Queue kann dementsprechend von der Statistik-Task ausgelesen werden, ob es zu einem Transfer-Miss kam. Damit ein Transfer-Miss nicht immer bei z.B. nur einem noch bestehendem Transfer der Statistik-Task signalisiert wird, muss die Anzahl der Tokens in der `transferTokens` Queue einen bestimmten Threshold übersteigen. Dieser kann variabel eingestellt werden. Es kann kein fester Wert gesetzt werden, da z.B. ein oder mehrere laufende Transfers bei unterschiedlichen Tilegrößen unterschiedlich gewichtet sind. Die Gewichtung bei unterschiedlichen Tilegrößen entsteht dadurch, dass unterschiedlich große Bereiche des Bildes bei einem Transfer-Miss noch nicht auf dem Display neugerendert wurden. Daher wäre ein Transfer-Miss bei einer Tilegröße von 10 * 10 Pixeln nicht so kritisch wie ein Transfer-Miss bei einer Tilegröße von 80 * 80 Pixeln.

5.6.4 Statistik-Task

```
Function statisticTask(void *param)
Initialisiere Zeitvariablen
while true do
  if Deadline-Misses vorhanden then
    aktuelle Framerate um X heruntersetzen
    warte bis zu dem nächsten Ausführungszeitpunkt
  end if
  if Transfer-Miss vorhanden then
    aktuelle Framerate um Y heruntersetzen
    warte bis zu dem nächsten Ausführungszeitpunkt
  end if
  berechne den Durchschnitt der Ausführungszeiten
  berechne mögliche Framerate
  if mögliche Framerate höher als aktuelle Framerate + W then
    erhöhe die aktuelle Framerate leicht um höchstens Z
  end if
end while
```

Algorithmus 5.3: Ablauf der Statistik-Task

Die Statistik-Task verarbeitet die gesammelten Performancedaten und entscheidet basierend auf diesen, welche Framerate von der Game-Engine angestrebt werden soll. Wird der Statistik-Task ein Transfer-Miss signalisiert, wird die aktuelle Framerate sofort um einen einstellbaren Wert X heruntergestuft und weitere Performancedaten werden nicht untersucht. Wird mit der Anzahl an Deadline-Misses ein einstellbarer Threshold überschritten, wird auch hier die FPS durch einen einstellbaren Wert Y gesenkt. Kommen Deadline-Misses auf, werden auch keine weiteren Performancedaten berücksichtigt. Die Werte X und Y sollten bestenfalls nach dem Empfinden des Entwicklers eingestellt werden, damit sich das Anpassen möglichst flüssig anfühlt. Setzt man diese z.B. zu hoch an, kann es zu Jitter kommen, wenn die Framerate aufgrund von Transfer- oder Deadline-Misses angepasst wird. Die Framerate wird beim Senken durch ein Minimum begrenzt, welches eingestellt werden kann. Das Minimum sorgt dafür, dass ein Spiel nicht „einfriert“. Weitere Performancedaten werden in beiden Fällen nicht untersucht, da die Game-Engine die aktuelle Framerate praktisch nicht unterstützen kann und weitere Performancedaten keinen Mehrwert zur Entscheidung der angestrebten Framerate mit sich bringen würden.
Kommt es zu keinen Deadline- oder Transfer-Misses, werden aus den letzten N Werten der Ausführungszeiten die durchschnittliche Ausführungszeit eines Frames berechnet. Die Anzahl N kann hier auch eingestellt werden. Erfahrungsgemäß aus der Implementierung der Game-Engine wurde die Zahl N abhängig von der Periodenzeit der Statistik-Task eingestellt. Die Implementierung der Game-Engine ermöglicht es, die Periodenzeit einzustellen, damit man möglichst variabel je nach Bedarf die Framerate anpassen kann. Dementsprechend sollte man N unterschiedlich einstellen, um nur die aktuellesten Messungen je Statsitik-Task Job zu analysieren. Beim Implementieren hat sich für eine Periodenzeit von 1 Sekunde das Analysieren der letzten 20 gemessenen Ausführungzeiten als effektiv herausgestellt. Aus der durchschnittlichen Ausführungszeit einer gesamten Periode wird die mögliche Framerate berechnet. Hierbei wird noch ein einstellbarer Puffer mit einbezogen, der beschreibt, welchen Anteil eines Frames die addierte Ausführungszeit der Render-Task und Logic-Task einnehmen. Ein Puffer muss miteinbezogen werden, da neben den beiden Tasks weitere Task und auch das Echtzeitbetriebssystem Rechenzeit einnehmen. Würde man keinen Puffer mit in die mögliche Framerate einbeziehen, würde man eine zu hohe mögliche Framerate berechnen, die direkt zu Überlastung führen könnte.
Ist die mögliche Framerate kleiner als die aktuelle Framerate, wird die Framerate erstmal nicht heruntergestuft. Grund dafür ist, dass die Game-Engine praktisch diese Framerate einhalten kann und der Puffer somit ein wenig zu großzügig gewählt wurde. Ist die mögliche Framerate höher als die aktuelle, wird die aktuelle höhergestuft. Damit es nicht zu großen Sprüngen beim Anpassen kommt, was praktisch bei der Entwicklung zu Bildflackern geführt hat, wird die Framerate höchstens in einstellbaren Schritten der Größe Z erhöht. Während der Implementierung hat sich ein Wert 5 als praktisch erwiesen. Außerdem kam ein Bildflackern bei stetigem Wechseln zwischen zwei Frameraten auf. Daher

wird auch auf die höhere Framerate nur gewechselt, wenn diese einen einstellbaren Wert W höher ist, als die aktuelle Framerate. Erfahrungsgemäß trat kein Bildflackern auf, wenn die Framerate nur angepasst wurde, wenn die Differenz der neuen möglichen Framerate und der aktuellen Framerate größer als 1 war.
Damit die Framerate nicht gegen unendlich laufen kann und somit die Game-Engine zum Abstürzen bringen kann, wird die Framerate durch ein einstellbares Maximum begrenzt.
Die Priorität der Statistik-Task wird auf 6 gesetzt. Somit ist dies die Task mit der höchsten Priorität in der Game-Engine. Grund dafür ist, dass die Statistik-Task immer von dem Scheduler ausgeführt wird, wenn diese Ready ist. Damit wird erreicht, dass die Statistik-Task auch bei Überlast immer Rechenzeit zugeteilt bekommt, um dann die Überlast zu regeln.

5.7 Schnittstelle zur Spieleprogrammierung

Damit der Nutzer losgelöst von dem Engine-Kern Spiele implementieren kann, wird in der Game-Engine dafür eine Schnittstelle bereitgestellt. Umgesetzt wird die Schnittstelle durch die Klasse `World`. Hauptaufgaben dieser Klasse sind, die Funktionen der Engine bereitzustellen und die Möglichkeit zu geben, die Spielelogik zu implementieren. Damit der Nutzer Zugriff auf Features wie z.B. die TileMap (siehe Kap. 5.3.2) oder den Audioservice (siehe Kap. 5.5.2) hat, stellt die Klasse `World` Instanzen der `TileMap`, des `InputService` und des `AudioService` zur Verfügung.
Um die Spielelogik implementieren zu können, werden durch die Klasse `World` die Funktionen `init()` und `update(uint64_t deltaTime)` zur Verfügung gestellt. Die Funktion `init()` dient zur Initialisierung des Spiels. Hierbei können z.B. zum Start des Spiels alle Spieleobjekte (siehe Kap. 5.8) erstellt oder das Design der `TileMap` festgelegt werden. Die Funktion wird nur einmalig zu Beginn des Spieles aufgerufen, um das Spiel somit vorzubereiten.
Die Funktion `update(uint64_t deltaTime)` enthält die eigentliche Spielelogik und wird je Logic-Task Job einmal aufgerufen. In dieser Funktion kann z.B. auf Eingaben reagiert werden, Abspielen von Sounds getriggert werden oder Kollision geprüft werden. Es wird keine feste Struktur zur Implementierung innerhalb der `update(uint64_t deltaTime)` Funktion gegeben, damit man möglichst viele unterschiedliche Spiele implementieren kann. Der Parameter `uint64_t deltaTime` nimmt die vergangene Zeit seit der letzten Ausführung der `update(uint64_t deltaTime)` entgegen, welche durch die Logic-Task (siehe Kap. 5.2) ermittelt wird. Diese Zeit kann dann in der Spieleimplementierung genutzt werden, um die Spielelogik losgelöst von der Periodenzeit eines Frames ausführen zu können.
Im Kapitel 6 wird die Implementierung von einem Beispiel-Spiel genauer erläutert, um den Umgang mit der Game-Engine und die Funktionsweise genauer darzustellen. Eine weitere Aufgabe der Klasse `World` ist es, den aktuellen Spielzustand nach einem Aktualisieren ein-

zufrieren. Hierfür wird die Funktion `createSnapShot()` bereitgestellt. Diese kopiert alle Tiles der TileMap und alle Spieleobjekte aus dem Objektregister in die Struktur `SnapShot`, welche dann von der Logic-Task an die Render-Task verschickt wird.
Das Objektregister dient als Verwaltungsstelle für alle im Spiel enthaltenen Spieleobjekte. Umgesetzt wird dieses über die Klasse `ObjectRegister`. Genutzt wird das Register vor allem, damit alle dort registrierten Spieleobjekte in den `SnapShot` mit berücksichtigt werden. Dadurch, dass der Nutzer die genutzten Spieleobjekte dort registrieren kann, fällt es weg, dass der Nutzer manuell die Erzeugung des `SnapShot` abhängig von den Inhalten des Spiels implementieren muss. Außerdem kann das `ObjectRegister` für weitere Funktionalitäten genutzt werden, welche später erklärt werden.

5.8 Spieleobjekte

Spieleobjekte dienen in der Game-Engine als Basiseinheit, welche bestimmte Daten bündeln und als technische Schnittstelle innerhalb der Engine genutzt werden. Umgesetzt werden die Spieleobjekte durch die Klasse `GameObject`. Die Klasse hält Daten wie die Position des Objekts innerhalb des Spiels oder die Textur des Objekts. Diese beiden Informationen alleine werden genutzt, um die `SnapShot` Generierung und das Rendering automatisch umzusetzen. Das heißt, dass der Nutzer nur passende Spieleobjekte in seiner Implementierung erstellen muss, um alle Inhalte, die er darstellen möchte, automatisch rendern lassen zu können. Hierbei müssen die Spieleobjekte nicht einmal bestimmte Logik ausführen und sie können nur für die Darstellung genutzt werden. Dennoch bieten sich die Spieleobjekte dafür an, bestimmte Features für die Game-Engine bereitzustellen. Im Rahmen dieser Arbeit wurden folgende Features bereitgestellt.

5.8.1 Kollision

Da Spieleobjekte bereits eine Position speichern, eignen diese sich dafür, automatische Kollisionserkennung bereitzustellen. Automatisch ist in diesem Sinne so gemeint, dass der Nutzer nicht selber die Überprüfungslogik, ob z.B. zwei Instanzen eines Spiels miteinander kollidieren, implementieren muss. Bereitgestellt wird die Kollisionslogik über die Klasse `Collider`, welche als Instanz von der Klasse `GameObject` gehalten wird. Die Klasse `Collider` dient wiederum als abstrakte Oberklasse um eine allgemeine Schnittstelle für Kollision bereitzustellen. Dadurch kann Kollisionserkennung für unterschiedliche Formen bereitgestellt werden, indem Unterklassen der Klasse `Collider` erstellt werden. In der Game-Engine wird die Kollisionserkennung für Rechtecke und für Kreise bereitgestellt durch die Unterklassen `BoxCollider` und `CircleCollider`. Somit kann einem Spieleobjekt eine Hitbox in Form eines Kreises oder eines Rechtecks zugewiesen werden. Zur Kollisionsüberprüfung werden die Funktionen `touchesAndOverlapsWith(Collider& otherCollider)` und `overlapsWith(Collider& otherCollider)` bereitgestellt. Diesen

kann der `Collider` eines anderen Objekts übergeben werden und es wird ein Wahrheitswert zurückgegeben, welcher angibt, ob eine Kollision stattfindet oder nicht. Beide Funktionen unterscheiden sich insofern, dass erstere nur überprüft, ob sich zwei Hitboxen überschneiden. Zweitere überprüft dazu noch, ob sich zwei Hitboxen berühren. Zudem werden über die Klasse `Collider` Kollisionstypen verwaltet. Diese Kollisionstypen werden zum einen dafür genutzt, eine allgemeinere Kollisionsabfrage bereitzustellen. Zum anderen werden diese genutzt, um eine Form der Bewegung umzusetzen, welche gestoppt wird, wenn mit einem vermerkten Kollisionstypen kollidiert wird. Verwaltet werden die Kollisionstypen über die Enumeration `COLLISION_TYPE`, welche vom Nutzer beliebig erweiterbar ist. Ein `Collider` kann hierbei seinen eigenen Typen und Typen, mit denen das Spieleobjekt bei Bewegung kollidieren soll, halten.
Dadurch, dass alle Spieleobjekte im Objektregeister hinterlegt werden müssen, kann man in Kombination mit den Kollisionstypen über die Klasse `World` anbieten, zu überprüfen, ob ein Objekt mit einem bestimmten Kollisionstypen kollidiert. Hierfür wird einfach über das Register iteriert und die Kollision mit jedem Objekt wird überprüft. Hierbei wird die Funktion über `World` angeboten, da diese Klasse das Register hält und somit Register und Objekte selber nicht miteinander verknüpft werden müssen.

5.8.2 Physik

Außerdem werden simple Physikeigenschaften, die auf Spieleobjekte ausgeübt werden können, zur Verfügung gestellt. Umgesetzt werden diese durch die Klasse `Physics`. Diese stellt bereit, dass zum einen eine Gravitationskraft auf ein Spielobjekt und zum anderen eine festgelegte Kraft für eine bestimmte Zeit auf ein Spielobjekt wirken kann. Beide Kräfte führen im Endeffekt zu einer Geschwindigkeit, welche zur Bewegung eines Spieleobjekts führt.

5.8.3 Bewegung und Dirty-Tile-Markierung

Um gleichmäßige Bewegung über Zeit zu ermöglichen, wird durch die Klasse `GameObject` eine Bewegungsrichtung und eine Geschwindigkeit gehalten. Weitere Bewegungsarten können ohne weitere Informationen bereitgestellt werden. Die Ausführung von Bewegungen selber wird nicht durch die Klasse `GameObject`, sondern durch die Schnittstelle `World` umgesetzt. Grund hierfür ist das Dirty-Tile Rendering. Kommt es zu Änderungen im Bild des Spiels, müssen die Teilbereiche, welche sich ändern, als neuzuzeichnen markiert werden. Fälle, in denen es dazu kommt, sind Bewegungen bzw. Positionswechsel und Texturänderungen. Diese beiden Fälle werden von der Game-Engine berücksichtigt und behandelt. Dies wird von der Schnittstelle `World` übernommen, da sie die TileMap bereitstellt und innerhalb der Spielelogik durch das Objektregister die Spieleobjekte eines Spiels verwaltet werden. Somit bleiben die Klassen `GameObject` und `TileMap` losgelöst voneinander und

es entsteht keine starke Vernetzung. Das Überprüfen, welche Teilbereiche neu gezeichnet werden müssen, wird im Falle einer Bewegung in zwei Schritte aufgeteilt. Vor der Bewegung werden alle Tiles der alten Position markiert, damit ggf. das Objekt dort nicht mehr gerendert wird. Nach der Bewegung werden alle Tiles der neuen Position markiert, um das Spieleobjekt endgültig zu rendern. Im Falle einer Texturänderung werden die Tiles an der Position des Objekts einmalig markiert. Sollte es beim Implementieren eines Spieles zu weiteren Fällen kommen, bei denen Tiles markiert werden müssen, kann dies über die Funktion `triggerTileMapToCheckRedraw(GameObject* o)` vom Nutzer manuell ausgeführt werden.
Insgesamt werden drei Bewegungsarten zur Verfügung gestellt: Spieleobjekte können zu einer bestimmten Position bewegt werden, deren Position kann um einen bestimmten Wert geändert werden und eine gleichmäßige Bewegung basierend auf der gehaltenen Geschwindigkeit und Bewegungsrichtung kann ausgeführt werden. Bei der gleichmäßigen Bewegung wird zudem bereitgestellt, eine Bewegung mit Kollisionsüberprüfung (siehe Kap. 5.8.1) auszuführen. Hierbei wird ein Spieleobjekt nur soweit bewegt, bis es mit einem Spieleobjekt kollidiert, welches einen Kollisionstypen hält, der beim bewegten Spieleobjekt als blockierend vermerkt ist. Außerdem wird bei der gleichmäßigen Bewegung nur so viel bewegt, wie Zeit seit der letzten Spiele-Aktualisierung vergangen ist.
Möchte man die Klasse `GameObject` um spielespezifische Funktionalitäten erweitern, kann man hier das Konzept der Vererbung nutzen. Beispielsweise kann man dann weitere Datentypen oder Funktionen hinzufügen. Da die Klasse `GameObject` sehr dünn gehalten ist, gibt es keine viele strukturelle Einschränkungen, inwiefern diese erweitert werden kann. Somit sind auch unterschiedliche Implementierungen von Spielen mit unterschiedlichen Designs möglich.

5.9 Init-Task

Die Init-Task dient der Initialisierung des Spiels, der Texturen und Sounds. Die Init-Task wurde dem Design hinzugefügt, da es bei der Implementierung zu Situationen kam, wo Spieleobjekte vor einer Textur initialisiert wurden, welche sie eigentlich benutzt haben. Dadurch wurden die Spieleobjekte nicht gerendert. Dies lag daran, dass Spieleobjekte, Texturen und Sounds als globale Objekte über unterschiedliche Dateien verwaltet werden. Hierbei gibt es keine feste Initialisierungsreihenfolge und ein Objekt kann somit vor seiner Textur erzeugt werden. Die Init-Task setzt hierbei die Reihenfolge so fest, dass alle Assets vor dem Spiel und somit den Spielobjekten initialisiert werden. Die Task läuft nur einmalig beim Start der Game-Engine.

6 Proof of Concept

Als Proof of Concept und zur Vorbereitung der Evaluation wurde ein Spiel mit der Game-Engine implementiert. Im Folgenden wird kurz erläutert, wie die Game-Engine hierfür genutzt wurde.

6.1 Flatter Vogel

Für den Flappy-Bird-Klon, hier Flatter Vogel, wurden drei Texturen erstellt. Bei den Texturen handelt es sich um eine für den Vogel, eine für die Röhren und eine für den Hintergrund. Bei der Vogel-Textur wurde die Funktion der Bitmaske genutzt, um keinen farbigen, eckigen Rand um den Vogel zu haben.
Für Vogel und Röhren wurden insgesamt siebeb Spieleobjekte erstellt. Dem Spieleobjekt, welches den Vogel in der Implementierung darstellt, wird die Vogel-Textur sowie ein `BoxCollider` zugewiesen. Die Röhren werden durch die restlichen sechs Spieleobjekte dargestellt, wobei diese in jeweils zu Paaren zusammengefasst werden. Warum genau sechs Röhren genutzt werden, wird später erläutert. Auch diesen wird die passende Textur und ein `BoxCollider` zugewiesen. Zum Schluss wird jeder Tile der Tilemap die Hintergrund-Textur zugewiesen. Bei der Implementierung werden Tiles der Größe 20 mal 20 Pixeln genutzt. Die Initialisierung und Zuweisung geschieht in der Funktion `init()`.
Die eigentliche Spielelogik wird in der Funktion `update(uint64_t deltaTime)` implementiert. Hierbei wird je Durchlauf zuerst abgefragt, ob der „A“-Knopf gedrückt wurde. Wenn ja, wird auf den Vogel eine Kraft für 30 ms nach oben wirken gelassen. Damit sich die Kräfte alter Sprünge nicht mit der neuen Kraft vermischen und verstärken, werden diese zurückgesetzt. Damit der Vogel nach unten fällt, wirkt eine Gravitationskraft nach unten auf dieses Spieleobjekt. Damit diese den Sprung nicht ausgleicht, muss sie auch vor dem Ausführen des Sprunges zurückgesetzt werden. Wurde die Eingabe überprüft, werden aus den wirkenden Kräften die Geschwindigkeiten berechnet und der Vogel wird passend dazu bewegt. Zudem werden die Röhren mit einer konstanten Geschwindigkeit bewegt. Es werden genau sechs Röhren genutzt, um einen Endlos-Effekt zu erzeugen. Diese wurden so positioniert, dass immer nur maximal zwei Röhren-Paare auf dem Bildschirm zu sehen sind. Dadurch kann das nicht zu sehende Röhren-Paar immer hinten drangehängt werden. Ein Paar wird hinten drangehängt, wenn es den linken Bildschirmrand verlässt. Außerdem wird je Durchlauf überprüft, ob der Vogel mit einer der Röhren kollidiert. Wenn ja, wird

das Spiel gestoppt und es kann neu gestartet werden. Zu dem wird geprüft, ob der Vogel ein Röhren-Paar passiert hat. Hierbei wird je Röhre eine Flag `passed` gehalten. Um dies umsetzen zu können, wurde die Klasse `GameObject` durch Vererbung zur Klasse `Pipe` erweitert, um die Flag zu halten. Der Vogel hat ein Röhren-Paar passiert, wenn sich dieser rechts von einem Paar befindet. Für jedes passierte Paar wird im Spiel der Score erhöht. Die Flag ist dafür da, um zu markieren, dass ein Paar bereits passiert wurde. Dadurch wird nicht je Durchlauf der Score erhöht, nur weil ein Röhren-Paar links vom Vogel ist. Die Flag wird beim Hintendranhängen wieder zurückgesetzt. Der Score wird bei Kollision mit einer Röhre über den Serial-Monitor ausgegeben.
Die Abbildung 6.1 stellt einen Ausschnitt des Spiels laufend auf dem fertigen Handheld dar.

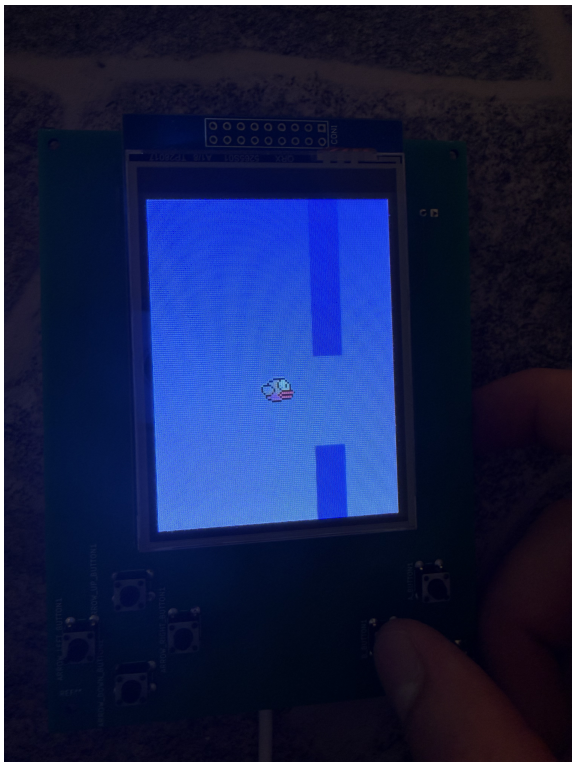


Abbildung 6.1: Ausschnitt des Spiels

7 Evaluation

Um die Game-Engine zu evaluieren, wurden mehrere Messungen ausgeführt. Die Ausführung der Messungen und die Ergebnisse werden im Folgenden dargestellt.

7.1 Evaluations-Setup

Für die Evaluation der Game-Engine wurde die in Hardware-Kapitel 4.1 vorgestellte Hardware genutzt. Hierbei waren die Hardware-Komponenten bereits auf der PCB verbaut. Die Game-Engine wurde mit der ESP-IDF-Version v6.0 gebaut und geflasht. Die genutzte Version des FreeRTOS Kernels war V10.5.1. Die CPU-Clock-Rate war während der Messungen auf 360 MHz eingestellt, die FreeRTOS-Tick-Frequenz betrug 1000 MHz. Zum Messen der Zeiten wurde die Funktion `esp_timer_get_time()` verwendet [5], da diese Funktion mit einer Auflösung von 1 Mikrosekunde möglichst genaue Zeitmessungen ermöglicht.

7.2 Task-Ansatz vs. Main-Loop

Neben der Implementierung der Game-Engine wurde zum Evaluieren ein naiver Main-Loop-Ansatz implementiert. Hierbei wurde das gesamte Spiel manuell in einer Main-Loop programmiert. Dazu gehört auch, dass der Input über Polling abgefragt wurde und, dass das Rendern manuell umgesetzt werden musste und nicht wie in der entwickelten Engine automatisch übernommen wird. Um die Performance und Robustheit beider Ansätze zu vergleichen, wurde die Framezeit bei beiden Ansätzen über eine Zeitspanne gemessen. Um die Framezeiten möglichst genau zu messen, wurde bei der Game-Engine die Zeit zwischen zwei Render-Task-Jobs gemessen, bei der Main-Loop wurde die Zeit zwischen zwei Iterationen gemessen. Insgesamt wurden je Ansatz zwei Messungen mit drei Durchläufen für zwei Minuten durchgeführt. Beim Messen wurde Flappy Bird manuell gespielt. Bei der Game-Engine war die dynamische Framerate-Anpassung aktiviert. Diese war wie folgt eingestellt:
Der Start-FPS-Wert betrug 30 FPS (`STARTFPS` = 30.0f) und die Statistik-Task wurde mit einer Periodenzeit von 1 Sekunde (`STATSPERIODINMS` = 1000) ausgeführt. Die minimal erreichbare Framerate betrug bei der Messung 20 FPS (`MIN_FPS` = 20.0f) und die Maximale 120 FPS (`MAX_FPS` = 120.0f). Die Statistik-Task hat immer die letzten 20 gemessenen Ausführungszeiten (`STATSVALUESCOMPARECOUNT` = 20) zur Berechnung

der aktuell möglichen Framerate genutzt. Zudem wurde der Anteil für den Puffer innerhalb der Framezeit auf 20 % gesetzt (`UTALIZATION_BUFFER` = 0.2f). Die Framerate wurde verändert, wenn sie sich schon um den Wert 1 (`FPS_THRESHOLD` = 1) von der alten unterschieden hat, maximal wurde die aktuelle Framerate um den Wert 5 verändert (`FPS_MAX_OFFSET_PER_ITERATION` = 5). Die Statistik-Task hat bei einer Anzahl von fünf Deadline-Misses (`DEADLINE_MISS_THRESHOLD` = 5) mit einer Frameratesenkung von zwei FPS (`FPS_DECREASE_ON_DEADLINE_MISS` = 2) reagiert. Ein Transfer-Miss wurde registriert, wenn fünf Transfers (`TRANSFER_MISS_THRESHOLD` = 5) offen waren, als der nächste Render-Job gestartet ist, reagiert wurde mit einer Frameratesenkung von 2 FPS (`FPS_DECREASE_ON_TRANSFER_MISS` = 2).
Die Tile-Größe war während der Messungen auf 20 * 20 Pixel eingestellt.
Während der ersten Messung wurde die Performance beider Ansätze ohne zusätzliche Last gemessen, um somit eine Vergleichsbasis zu erlangen. In der zweiten Messung wurde sowohl in der Game-Engine als auch bei dem naiven Schleifen-Ansatz eine Task hinzugefügt, welche Nebenlast erzeugt. Hierfür wurde eine Task mit Priorität 1 erstellt, welche eine Zufallszahl N zwischen 10000 und 100000 erzeugt und dann N mal eine Fließkommazahlberechnung ausführt. Die Task wurde mit einer Periodenzeit von 50 ms ausgeführt. Auch bei der Implementierung der Main-Loop konnte die Task zur Erzeugung der Nebenlast genutzt werden, da die Ausführung der Main-Loop innerhalb der ESP-IDF durch die Main-Task gestartet wird und dementsprechend auch durch diese ausgeführt wird [9]. Zu beachten ist, dass die Implementierung der Game-Engine und der naiven Main-Loop nicht dieselben Features haben. Die Game-Engine unterstützt beispielsweise Audio und die dynamische Framerate-Anpassung, die Main-Loop nicht. Außerdem nutzt die Game-Engine das Tile-Rendering, die Main-Loop rendert den gesamten Bildschirm

7.2.1 Messergebnisse

Die folgenden Messergebnisse sind zustande gekommen. In dieser Arbeit wird je Messung nur ein Durchlauf dargestellt, da sich die Durchläufe ähnlich verhalten haben.
Abbildung 7.1 stellt einen Messdurchlauf ohne Nebenlast dar. Hierbei repräsentiert die X-Achse die vergangene Messzeit in Millisekunden, die Y-Achse die über die Messzeit gemessenen Framezeiten in Millisekunden. Der blaue Graph zeigt die Framezeiten der Game-Engine-Implementierung, der rote Graph repräsentiert die Framezeiten der Main-Loop-Implementierung. Aus den Framezeiten ist zu erkennen, dass der Main-Loop-Ansatz eine geringere Framezeit als die Game-Engine erreicht hat, also damit eine höhere Framerate erreichte. Außerdem ist zu erkennen, dass es bei der Game-Engine Implementierung zu vielen Peaks gekommen ist.
In Abbildung 7.2 wird ein Messdurchlauf mit Nebenlast dargestellt. Auch hier spiegelt die X-Achse die vergangene Messzeit in Millisekunden, die Y-Achse die über die Messzeit ge-

messenen Framezeiten in Millisekunden wieder. Der blaue Graph zeigt die Framezeiten der Game-Engine Implementierung, der rote Graph repräsentiert die Framezeiten der Main-Loop Implementierung. Hier ist zu erkennen, dass die Game-Engine Implementierung eine niedrigere Framezeit als die Main-Loop Implementierung erreicht hat. Außerdem ist an den Peaks der Framezeiten der Main-Loop Implementierung zu vielen Schwankungen der Framezeiten gekommen ist.

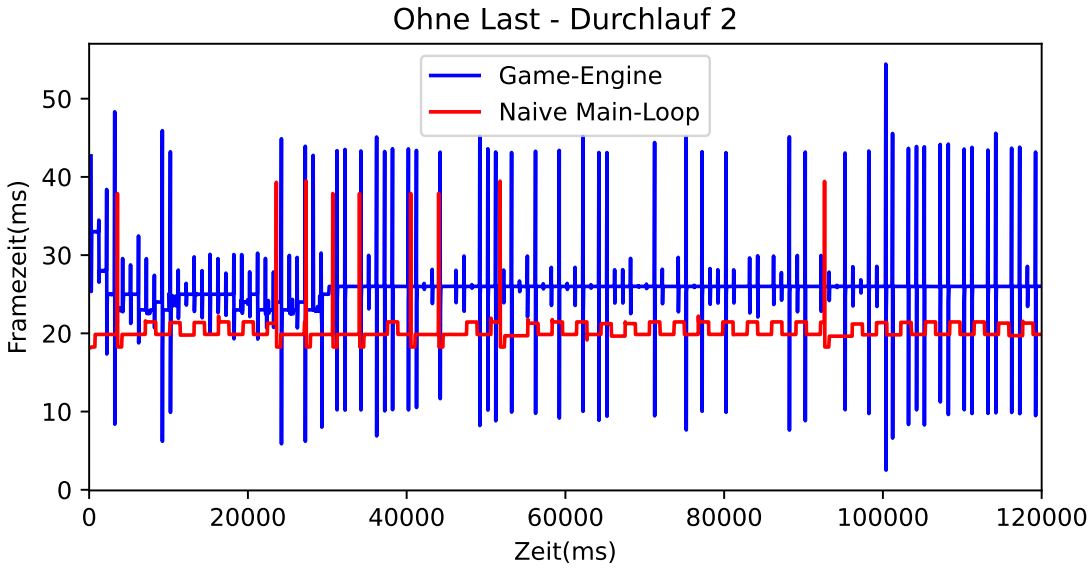


Abbildung 7.1: Durchlauf ohne Last

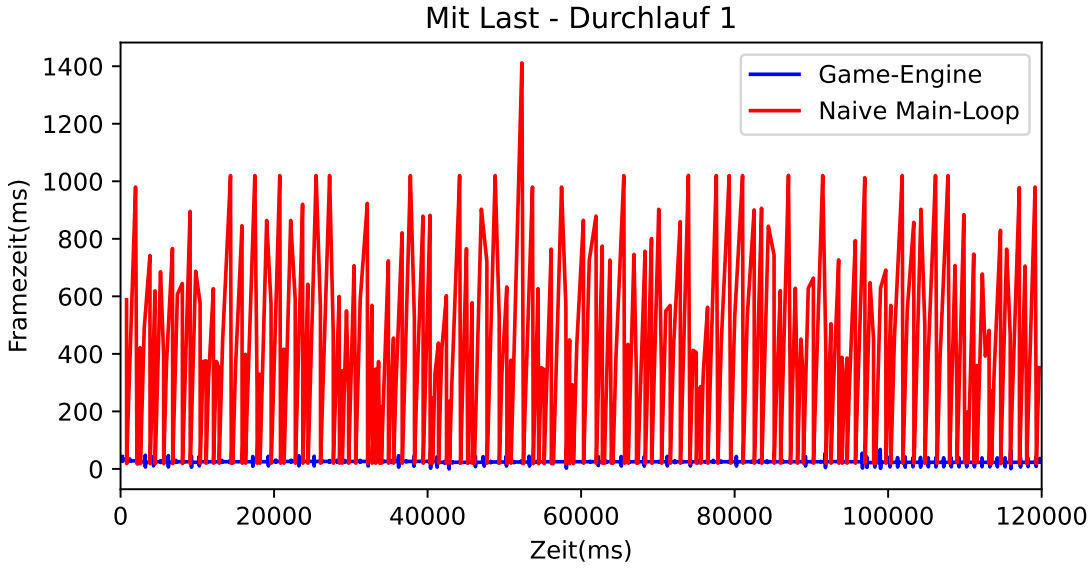


Abbildung 7.2: Durchlauf mit Last

Für die Messung ohne Last hat die Game-Engine eine Durchschnitts-Framezeit von 25,32 ms, die Main-Loop 19,86 ms. Die Standardabweichung der Game-Engine beträgt 3,15, die der Main-Loop 1.08.
Für die Messung mit Nebenlast hat die Game-Engine eine Durchschnitts-Framezeit von 25,13 ms erreicht, die Main-Loop 304,18 ms. Die Standardabweichung der Game-Engine beträgt 3,78, die der Main-Loop 332,03.

7.2.2 Auswertung

Die Messung ohne Last zeigt, dass der Ansatz der naiven Main-Loop etwas performanter ist, als der Ansatz mit der Game-Engine, da die Loop eine Framerate von ca. 50 FPS und die Game-Engine nur eine Framerate von ca. 39 FPS erreicht. Betrachtet man jedoch in den Einstellungen der dynamischen Framerate-Anpassung, dass 20 % der summierten Ausführungszeiten als Puffer in die Framerate miteingebunden wird, würde die Game-Engine ohne diesen Puffer auch eine Framerate von ca. 50 FPS erreichen. Besonders auffällig sind die großen Peaks in den Framezeiten der Game-Engine. Diese entstehen wahrscheinlich dadurch, dass sich die Last aufgrund des Tile-Renderings je Frame unterscheiden kann, ungünstige Interrupts durch Eingaben aufkommen oder aufgrund der dynamischen Framerate-Anpassung Deadline-Misses erzwungen werden, sodass das Rendering für kurze Zeit preempted wird. Der Main-Loop-Ansatz läuft so stabil, da die Last je Frame identisch bleibt. Betrachtet man die Messung mit der erzeugten Nebenlast, erkennt man den klaren Vorteil des RTOS-Ansatzes für die Game-Engine im Vergleich zu der naiven Main-Loop bezüglich der Robustheit. Die Performance der Game-Engine ändert sich kaum im Vergleich zu der Messung ohne Nebenlast. Grund dafür ist, dass der FreeRTOS Scheduler der erzeugten Task nur freie Rechenzeit zuordnet, wenn es Freiraum gibt, und aufgrund der höheren Prioritäten der Game-Engine-Tasks der gesamten Game-Engine Vorrang in der Ausführung gibt. Bei dem Main-Loop Ansatz fehlt diese „Schutzstruktur“, was dazu führt, dass die Framerate komplett zusammenbricht. Mit einer durchschnittlichen Framezeit von 304.18 ms erreicht man eine Framerate von ca. 3 FPS. Mit solch einer Framerate ist das Spiel garcnicht spielbar gewesen.

7.3 Statische und dynamische Framerates

Um zu beurteilen, inwiefern sich die in dieser Arbeit umgesetzte dynamische Framerate-Anpassung auf das Spielerlebnis auswirkt, wird das Framezeit-Verhalten zu unterschiedlichen Start-FPS zwischen statischer und dynamischer Framezeit gemessen. Hierfür werden fünf unterschiedliche Start-FPS-Werte je zwei Durchläufe für zwei Minuten gemessen. Um das Verhalten bei statischen FPS zu messen, wird die dynamische Framerate-Anpassung deaktiviert. Bei Messungen mit der dynamischen Framerate-Anpassung wird mit folgenden Einstellungen gemessen: Der Start-FPS-Wert betrug 30 FPS (`STARTFPS` = 30.0f) und

die Statistik-Task wurde mit einer Periodenzeit von 1 Sekunde (`STATSPERIODINMS` = 1000) ausgeführt. Die minimal erreichbare Framerate betrug bei der Messung 20 FPS (`MIN_FPS` = 20.0f) und die Maximale 120 FPS (`MAX_FPS` = 120.0f). Die Statistik-Task hat immer die letzten 20 gemessenen Ausführungszeiten (`STATSVALUESCOMPARECOUNT` = 20) zur Berechnung der aktuell möglichen Framerate genutzt. Zudem wurde der Anteil für den Puffer innerhalb der Framezeit auf 20 % gesetzt (`UTALIZATION_BUFFER` = 0.2f). Die Framerate wurde verändert, wenn sie sich schon um den Wert 1 (`FPS_THRESHOLD` = 1) von der alten unterschieden hat, maximal wurde die aktuelle Framerate um den Wert 5 verändert (`FPS_MAX_OFFSET_PER_ITERATION` = 5). Die Statistik-Task hat bei einer Anzahl von fünf Deadline-Misses (`DEADLINE_MISS_THRESHOLD` = 5) mit einer Frameratesenkung von zwei FPS (`FPS_DECREASE_ON_DEADLINE_MISS` = 2) reagiert. Ein Transfer-Miss wurde registriert, wenn fünf Transfers (`TRANSFER_MISS_THRESHOLD` = 5) offen waren, als der nächste Render-Job gestartet ist, reagiert wurde mit einer Frameratesenkung von 2 FPS (`FPS_DECREASE_ON_TRANSFER_MISS` = 2). Die Tile-Größe war während der Messungen auf 20 * 20 Pixel eingestellt. Je Start-FPS Werte wurde die Config-Flag `STARTFPS` entsprechend eingestellt.

7.3.1 Messergebnisse

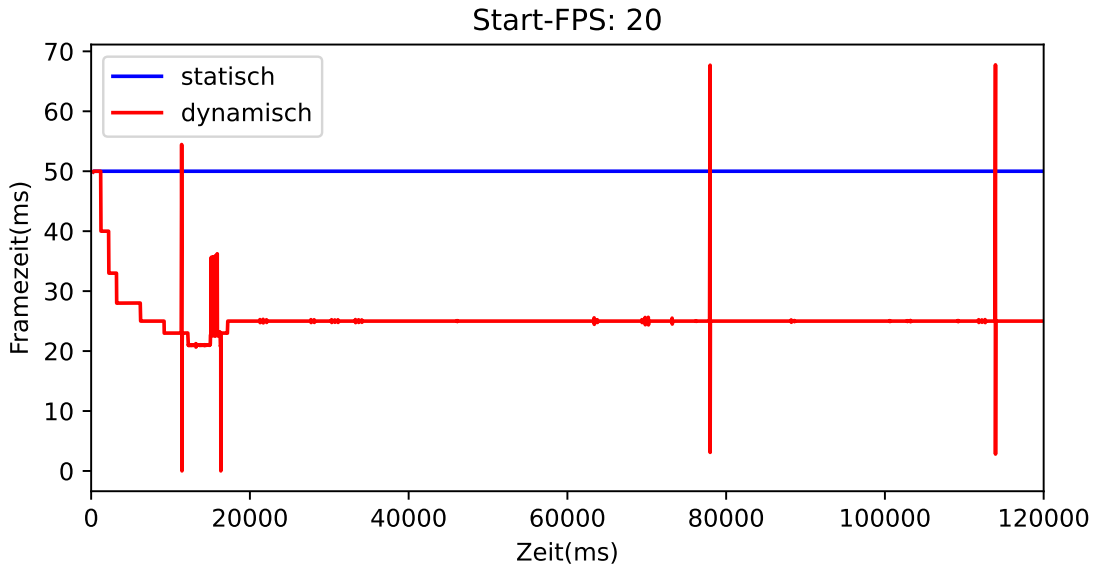


Abbildung 7.3: Durchlauf mit 20 FPS

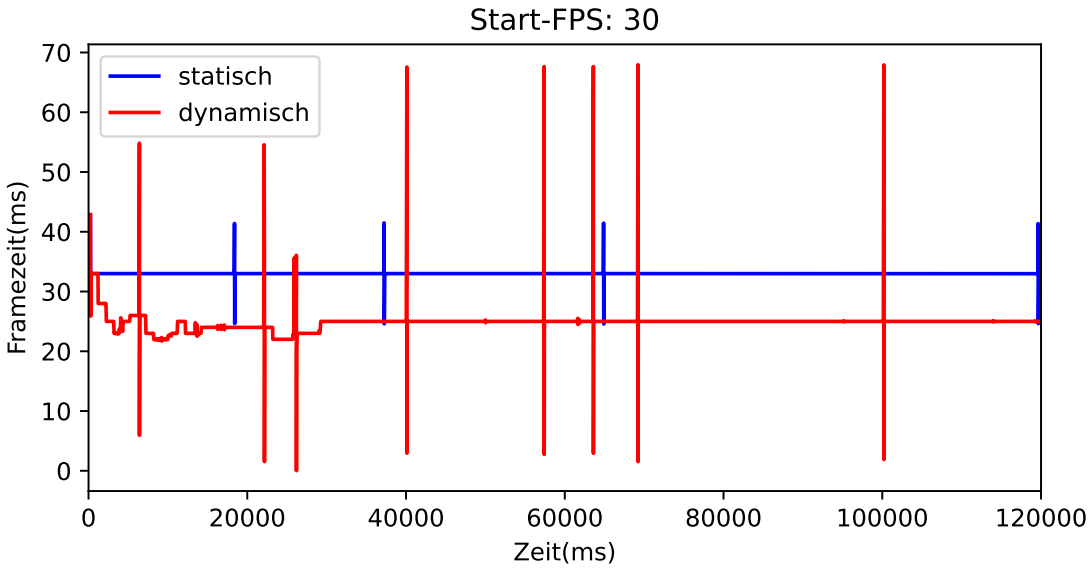


Abbildung 7.4: Durchlauf mit 30 FPS

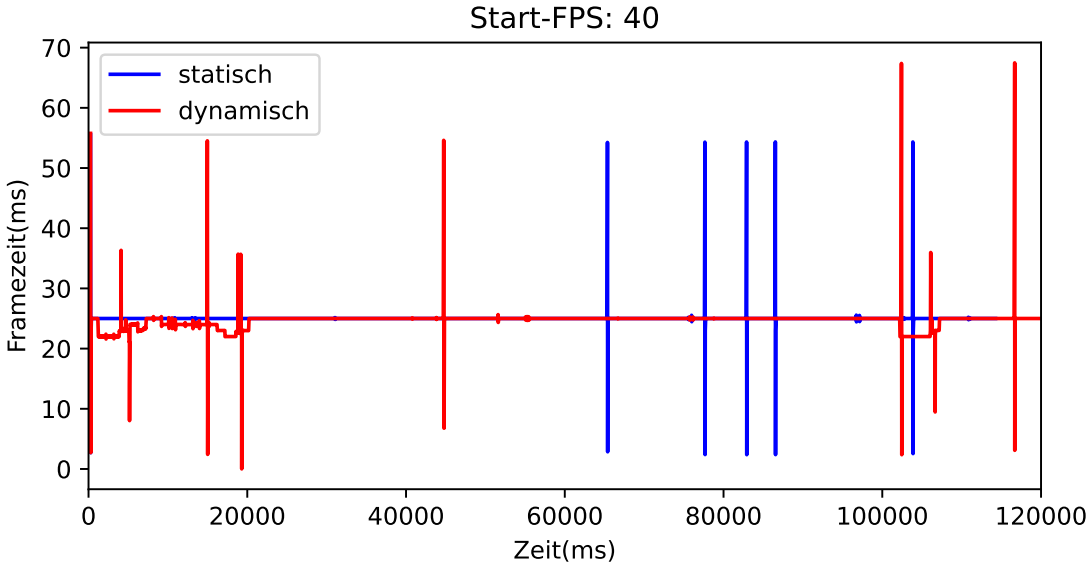


Abbildung 7.5: Durchlauf mit 40 FPS

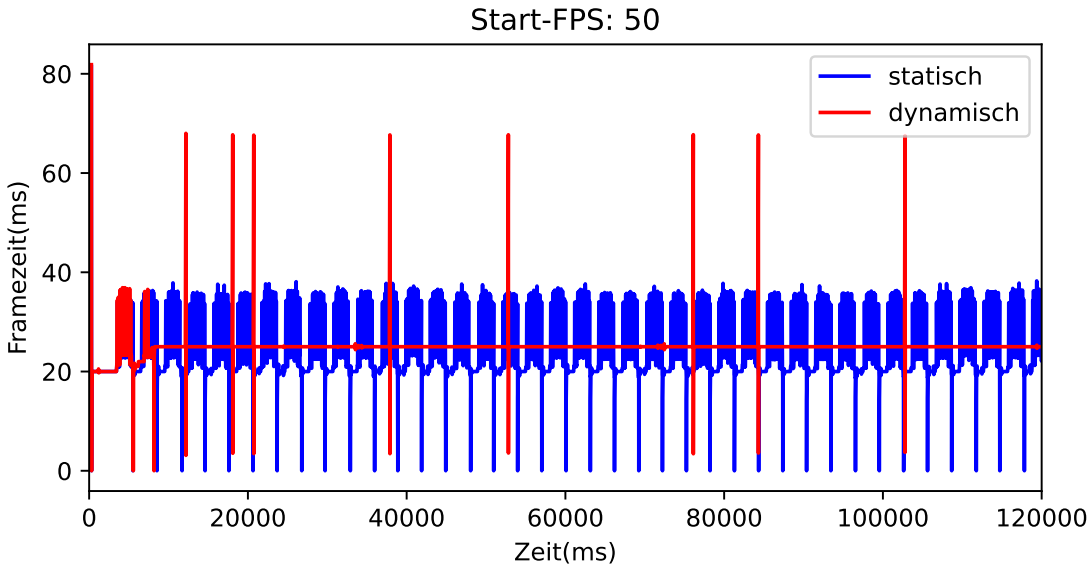


Abbildung 7.6: Durchlauf mit 50 FPS

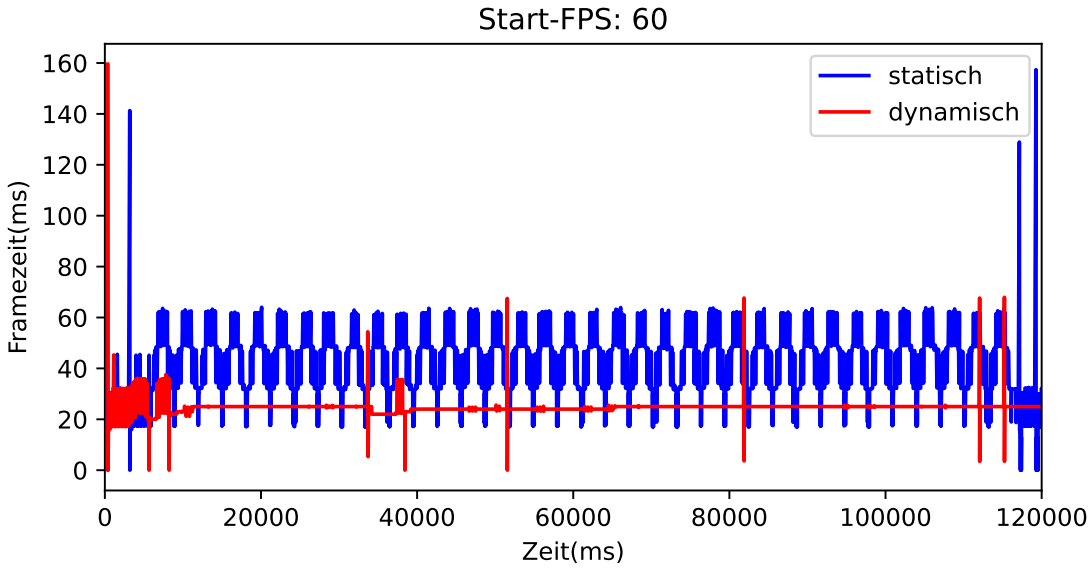


Abbildung 7.7: Durchlauf mit 60 FPS

Je Durchlauf haben sich die Messergebnisse sehr geähnelt. Je Start-FPS wird im Folgenden eine Messung grafisch dargestellt: Gemessen wurde für die Framerate-Werte 20 FPS (abb. 7.3), 30 FPS (Abb. 7.4), 40 FPS (Abb. 7.5), 50 FPS (Abb. 7.6) und 60 FPS (Abb. 7.7). Je Abbildung wird auf der X-Achse die vergangene Messzeit in Millisekunden, auf der Y-Achse die gemessenen Framezeiten in Millisekunden dargestellt. Der blaue Graph repräsentiert die Framezeiten ohne dynamische Framerate-Anpassung, der rote Graph stellt die Framezeiten mit dynamischer Framerate-Anpassung dar. Bei Frameraten bis 40 FPS

ist zu erkennen, dass die eingstellte Framerate ohne dynamische Framerate-Anpassung stabil gehalten wird. Ab einer Framerate von 50 FPS erkennt man bei den Messungen ohne dynamische Framerate-Anpassung viele Jitter und Peaks. Über alle Messungen mit dynamischer Framerate-Anpassung ist zu erkennen, dass sich die Framerate bei ca. 40 FPS einpendelt.

7.3.2 Auswertung

Die Messergebnisse stellen dar, dass die dynamische Framerate-Anpassung positiven Einfluss auf das Spielerlebnis des Nutzers hat. Dies ist an zwei unterschiedlichen Punkten zu erkennen. Betrachtet man z.B. den Verlauf für den Start-FPS-Wert 20, sieht man beim statischen Verlauf, dass dieser Wert konstant gehalten wird. Betrachtet man den Verlauf der dynamischen Framerate-Anpassung, sieht man, dass diese sich zuerst einpendelt und dann konstant bei einer Framerate von ca. 40 FPS bleibt. Das zeigt, dass die dynamische Framerate-Anpassung die vorhandene Rechenleistung möglichst gut ausnutzt, um ein möglichst flüssiges Bild zu bekommen. Die betrachtete Messung hier zeigt, dass der Nutzer ohne dynamische Framerate-Anpassung durchgängig nur mit 20 FPS spielen würde, was zu einem „ruckeligen“ Spielerlebnis führt. Das heißt, wird der Start-FPS-Wert zu niedrig angesetzt, passt sich die Game-Engine so an, dass sie die verfügbare Leistung möglichst auslastet.
Betrachtet man wiederum den Messverlauf für den Start-FPS-Wert 60 FPS, erkennt man, dass hier der Fall eintritt, dass man den Startwert deutlich zu hoch angesetzt hat. Ohne dynamische Framerateanpassung erkennt man, dass es immer wieder zu Peaks in den Framezeiten kommt, was als Ruckler und Jitter während des Spielens wahrgenommen wird. Mit dynamischer Framerate-Anpassung sieht man, dass man zu Beginn auch noch starke Peaks bei den Framezeiten hat. Dies liegt daran, dass sich die Anpassung erstmal einpendeln und die mögliche Framerate richtig einstellen muss. Nachdem sich die mögliche Framerate eingependelt hat, erkennt man, dass diese auch konstant von der Game-Engine gehalten wird. Das Einpendeln benötigte in den Messungen ca. 8 bis 10 Sekunden. Dementsprechend hat man keinen Jitter und keine vielen Peaks in den Framezeiten mehr und das Spiel kann wieder flüssig wahrgenommen werden.

7.4 Optimale Tilegröße

Um zu untersuchen, wie sich die Game-Engine bei unterschiedlichen Tilegrößen verhält, werden die Framezeiten bei unterschiedlichen Tilegrößen gemessen. Insgesamt werden fünf unterschiedliche Tilegrößen untersucht. Je Tilegröße wurden drei Durchläufe von je zwei Minuten gemessen. Es wurde wieder manuell Flappy-Bird gespielt und dynamische Framerate-Anpassung wurde mit folgenden Einstellungen ausgeführt: Der Start-FPS-Wert betrug 30 FPS (`STARTFPS` = 30.0f) und die Statistik-Task wurde mit einer

Periodenzeit von 1 Sekunde (`STATSPERIODINMS` = 1000) ausgeführt. Die minimal erreichbare Framerate betrug bei der Messung 20 FPS (`MIN_FPS` = 20.0f) und die Maximale 120 FPS (`MAX_FPS` = 120.0f). Die Statistik-Task hat immer die letzten 20 gemessenen Ausführungszeiten (`STATSVALUESCOMPARECOUNT` = 20) zur Berechnung der aktuell möglichen Framerate genutzt. Zudem wurde der Anteil für den Puffer innerhalb der Framezeit auf 20 % gesetzt (`UTALIZATION_BUFFER` = 0.2f). Die Framerate wurde verändert, wenn sie sich schon um den Wert 1 (`FPS_THRESHOLD` = 1) von der alten unterschieden hat, maximal wurde die aktuelle Framerate um den Wert 5 verändert (`FPS_MAX_OFFSET_PER_ITERATION` = 5). Die Statistik-Task hat bei einer Anzahl von fünf Deadline-Misses (`DEADLINE_MISS_THRESHOLD` = 5) mit einer Frameratesenkung von zwei FPS (`FPS_DECREASE_ON_DEADLINE_MISS` = 2) reagiert. Ein Transfer-Miss wurde registriert, wenn fünf Transfers (`TRANSFER_MISS_THRESHOLD` = 5) offen waren, als der nächste Render-Job gestartet ist, reagiert wurde mit einer Frameratesenkung von 2 FPS (`FPS_DECREASE_ON_TRANSFER_MISS` = 2).

7.4.1 Messergebnisse

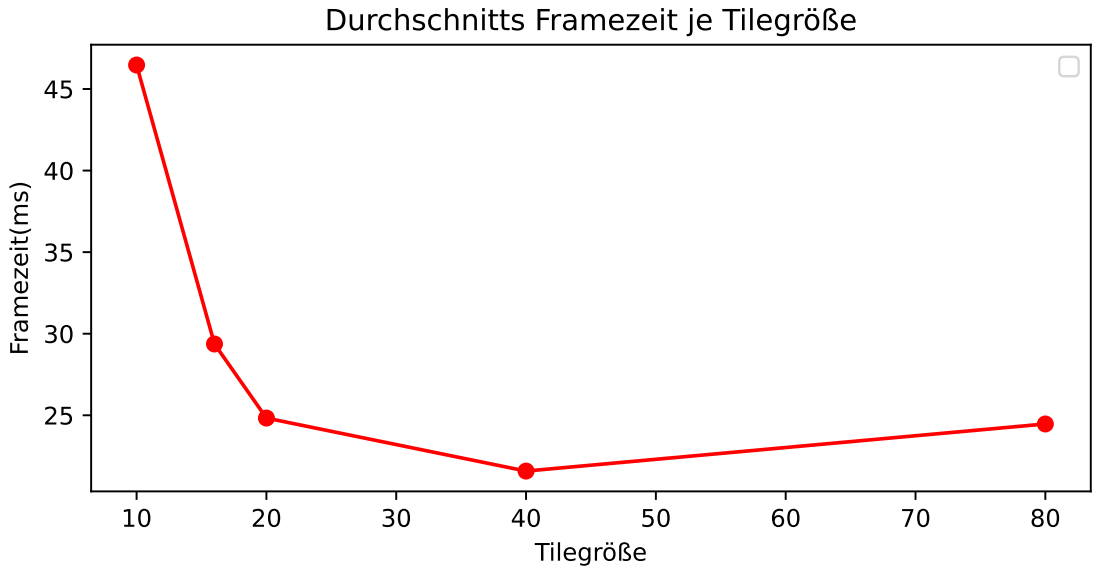


Abbildung 7.8: Framezeiten je Tilegröße

Abbildung 7.8 stellt die durchschnittlichen Framezeiten je Tile-Größe dar. Die X-Achse repräsentiert die Tile-Größen, die Y-Achse die Framezeiten in Millisekunden. Es ist zu erkennen, dass es ein Minimum der Framezeit bei einer Tile-Größe von 40 * 40 Pixeln gibt. Bei größeren und kleineren Tile-Größen werden die durchschnittlichen Framezeiten wieder größer. Für Tilegrößen kleiner als 10 * 10 Pixeln konnte keine Messung durchgeführt werden, da die Implementierung der Game-Engine nicht gestartet ist.

7.4.2 Auswertung

Der Verlauf der Durchschnittszeiten zeigt Vor- und Nachteile des Tile-Renderings. Bei Tile-Größen von kleiner als 20 * 20 Pixeln übertrumpft der Rechen- und Verwaltungsaufwand den Nutzen. Dies führt dazu, dass die Framezeiten größer werden und geringere Frameraten erreichbar sind. Bei Tilegrößen kleiner als 10 Pixeln startet die Game-Engine nicht, da der Speicherbedarf des Verwaltungsaufwandes zu hoch für den bereitgestellten Speicher des Microcontrollers ist. Je größer die Tilegröße wird, desto kleiner werden die Framezeiten, sodass auch höhere Frameraten erreicht werden. Das Optimum liegt bei einer Tilegröße von 40 * 40 Pixeln, wobei eine Framezeit von 21,58 und somit eine Framerate von ca 46 FPS erreicht wird. Bei zu hohen Tilegrößen wird die Framezeit wieder höher. Das liegt daran, dass deutlich mehr Daten verschickt werden müssen und der Bottleneck, welcher durch die Übertragungsbandweite des Bildschirms entsteht, Wirkung zeigt, da die Game-Engine so mehr Transfer-Misses erkennt und dementsprechend die Framerate runterregelt.

8 Zusammenfassung

Abschließend wird zusammengefasst, was in dieser Bachelorarbeit erreicht wurde und ein Ausblick für die Zukunft wird gegeben.

8.1 Was wurde erreicht?

Im Rahmen dieser Arbeit wurde erfolgreich eine Game-Engine basierend auf Konzepten aus Echtzeitbetriebssystemen entwickelt. Mit dieser können nun Spiele für Microcontroller entwickelt werden, wobei sich der Nutzer nur auf das Design des Spiels und dessen Logik fokussieren muss, ohne dass zum Beispiel das Rendering manuell implementiert werden muss. Außerdem wurde eine dynamische Framerate-Anpassung mit positivem Einfluss auf das Spieleerlebnis sowie ein physischer Handheld entwickelt, welcher als Spielesystem für die Game-Engine genutzt werden kann. Die Evaluation hat gezeigt, dass die Game-Engine robust ist, dennoch einen gewissen Overhead durch die Echtzeitbetriebssystem-Strukturen im Vergleich zu einer Implementierung mit einer Main-Loop mit sich bringt. Die gesetzten Anforderungen konnten in dieser Arbeit erfüllt werden, jedoch war das Testen unterschiedlicher Hardware, wie z.B. das Testen unterschiedlicher Bildschirme mit unterschiedlichen Interfaces, aufgrund der Zeit eingeschränkt.

8.2 Ausblick für die Zukunft

Die Game-Engine kann noch um viele Features erweitert werden. Beispielsweise kann man weitere Features auf der User-Ebene, wie zum Beispiel Animationen oder Level-Systeme, hinzufügen, um umfangreichere Spiele entwickeln zu können.
Zudem könnte man die Art des Renderings um Raycasting erweitern, um 3D-Spiele mit der Game-Engine entwickeln zu können. Damit es zu keinem Bildruckeln beim Anpassen der Framerate kommt, wird aktuell die Framerate nur schrittweise angepasst. Eine andere Möglichkeit, die Framerate möglichst flüssig anzupassen, wäre die Nutzung eines Proportional–Integral–Derivative (PID)-Controllers. Dies könnte dementsprechend auch in der Game-Engine eingebaut werden.

Abbildungsverzeichnis

3.1 Komponenten einer Game-Engine 7
3.2 Architektur der Game-Engine 9
3.3 Architektur der Hardware-Schicht 10
3.4 Architektur des Engine-Kerns 11
3.5 Architektur der User-Ebene 13

4.1 Schaltplan des Handhelds 17
4.2 Layout und 3D-Ansicht der PCB 19

5.1 Ablauf der Logic-Task 24
5.2 optimales Timing 29
5.3 Ablauf der Audio-Task 31
5.4 Periodenzeit ist zu groß 32
5.5 Deadline-Misses 33
5.6 Transfer-Misses 33

6.1 Ausschnitt des Spiels 42

7.1 Durchlauf ohne Last 45
7.2 Durchlauf mit Last 45
7.3 Durchlauf mit 20 FPS 47
7.4 Durchlauf mit 30 FPS 48
7.5 Durchlauf mit 40 FPS 48
7.6 Durchlauf mit 50 FPS 49
7.7 Durchlauf mit 60 FPS 49
7.8 Framezeiten je Tilegröße 51

Algorithmenverzeichnis

5.1 ISR für Hardware-Interrupt Verarbeitung 22
5.2 Ablauf der Render-Task 26
5.3 Ablauf der Statistik-Task 34

Quellcodeverzeichnis

5.1 Callback nach einem Transfer 26

Literatur

[1] Richard Barry und The FreeRTOS Team. *Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide*. Release Version 1.1.0. 2023. URL: https://www.freertos.org/Documentation/02-Kernel/07-Books-and-manual/01-RTOS_book (besucht am 02. 08. 2026).

[2] Jonathan Bartlett. „Basic Resistor Circuit Patterns“. In: *Electronics for Beginners: A Practical Introduction to Schematics, Circuits, and Microcontrollers*. Springer, 2020, S. 129–144.

[3] Giorgio Buttazzo. *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*. 4. Aufl. Springer, 2024.

[4] DFRobot. *FireBeetle 2 ESP32-P4 Edge AI Vision Board with MIPI & Wi-Fi 6 | DFRobot*. 2025. URL: https://www.dfrobot.com/product-2915.html (besucht am 29. 07. 2026).

[5] *ESP Timer (High Resolution Timer)*. Espressif Systems. URL: https://docs.espressif.com/projects/esp-idf/en/v6.0.2/esp32p4/api-reference/system/esp_timer.html (besucht am 30. 07. 2026).

[6] *ESP-IDF Programming Guide*. Espressif Systems. URL: https://docs.espressif.com/projects/esp-idf/en/v6.0.1/esp32p4/index.html.

[7] *ESP32-P4 Series Datasheet Pre-release v0.5*. Espressif Systems. URL: https://dfimg.dfrobot.com/62d52567aa9508d63a4247a1/wikien/30ca0837adaac2d5c26640d7b8e377fe.pdf (besucht am 12. 07. 2026).

[8] Mick Foley. „Advanced Graphics“. In: *Tricks of the Mac Game Programming Gurus*. Hrsg. von Jamie McCornack. Hayden Books, 1995, S. 121–208.

[9] *FreeRTOS Overview*. Espressif Systems. URL: https://docs.espressif.com/projects/esp-idf/en/v6.0.2/esp32p4/api-reference/system/freertos.html (besucht am 30. 07. 2026).

[10] *GPIO & RTC GPIO*. Espressif Systems. URL: https://docs.espressif.com/projects/esp-idf/en/v6.0.2/esp32p4/api-reference/peripherals/gpio.html (besucht am 01. 07. 2026).

[11] Jason Gregory. *Game Engine Architecture*. 3. Aufl. CRC Press, 2018.

[12] *Implications of Slow or Floating CMOS Inputs*. Texas Instruments. 2021. URL: https://www.ti.com/lit/an/scba004e/scba004e.pdf (besucht am 02. 08. 2026).

[13] *LCD*. Espressif Systems. URL: https://docs.espressif.com/projects/esp-idf/en/v6.0.2/esp32p4/api-reference/peripherals/lcd/index.html (besucht am 12. 07. 2026).

[14] *LED Control (LEDC)*. Espressif Systems. URL: https://docs.espressif.com/projects/esp-idf/en/v6.0.1/esp32p4/api-reference/peripherals/ledc.html (besucht am 20. 07. 2026).

[15] *LED Pattern Game with FreeRTOS on ESP32*. URL: https://github.com/OsaidHamza7/LED-Pattern-Game-with-FreeRTOS-on-ESP32 (besucht am 31. 07. 2026).

[16] Qing Li und Caroline Yao. *Real-Time Concepts for Embedded Systems*. CRC Press, 2003.

[17] Ansgar Meroth und Petre Sora. „Serial Peripheral Interface (SPI)“. In: *Sensornetzwerke in Theorie und Praxis: Embedded Systems-Projekte erfolgreich realisieren*. Springer Fachmedien Wiesbaden, 2021, S. 219–239.

[18] *Miscellaneous System APIs*. Espressif Systems. URL: https://docs.espressif.com/projects/esp-idf/en/latest/esp32p4/api-reference/system/misc_system_api.html (besucht am 29. 07. 2026).

[19] *PTS645 Series 6 mm Tact Switches*. C&K Switches. 2018. URL: https://assets.rs-online.com/v1700169123/Datasheets/faa3b5df6150436956cf2922026c8510.pdf (besucht am 24. 06. 2026).

[20] H. F. Schlaak. „Elektromechanische Relais“. In: *Handbuch der Mess-und Automatisierungstechnik im Automobil: Fahrzeugelektronik, Fahrzeugmechatronik*. Springer, 2006, S. 279–296.

[21] Abraham Silberschatz, Peter Baer Galvin und Greg Gagne. *Operating System Concepts*. 10. Aufl. Wiley, 2018.

[22] *Snake game using an STM32 board and an OLED screen*. URL: https://github.com/rsilverioo/STM32_snake (besucht am 31. 07. 2026).

[23] *The FreeRTOS™ Reference Manual*. Amazon Web Services. URL: https://www.freertos.org/media/2018/FreeRTOS_Reference_Manual_V10.0.0.pdf (besucht am 07. 07. 2026).

[24] *User Manual For 2.8"TFT Touch Shield for Arduino with Resistive Touch Screen (TF028)*. IDUINO. URL: https://asset.conrad.com/media10/add/160267/c1/-/en/001525436ML02/manual-1525436-display-suitable-for-arduino-board-arduino-uno-arduino.pdf (besucht am 22. 06. 2026).

Eidesstattliche Versicherung

(Affidavit)

Adolenko, Maximilian — 252750

Name, Vorname (surname, first name) — Matrikelnummer (student ID number)

■ Bachelorarbeit (Bachelor's thesis) ☐ Masterarbeit (Master's thesis)

Titel (Title)

Realisierung einer Game-Engine in eingebetteten Echtzeitbetriebssystemen

Ich versichere hiermit an Eides statt, dass ich die vorliegende Abschlussarbeit mit dem oben genannten Titel selbstständig und ohne unzulässige fremde Hilfe erbracht habe. Ich habe keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie wörtliche und sinngemäße Zitate kenntlich gemacht. Die Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

I declare in lieu of oath that I have completed the present thesis with the above-mentioned title independently and without any unauthorized assistance. I have not used any other sources or aids than the ones listed and have documented quotations and paraphrases as such. The thesis in its current or similar version has not been submitted to an auditing institution before.

Dortmund, 07.08.2026 — Adolenko

Ort, Datum (place, date) — Unterschrift (signature)

Belehrung:
Wer vorsätzlich gegen eine die Täuschung über Prüfungsleistungen betreffende Regelung einer Hochschulprüfungsordnung verstößt, handelt ordnungswidrig. Die Ordnungswidrigkeit kann mit einer Geldbuße von bis zu 50.000,00 € geahndet werden. Zuständige Verwaltungsbehörde für die Verfolgung und Ahndung von Ordnungswidrigkeiten ist der Kanzler/die Kanzlerin der Technischen Universität Dortmund. Im Falle eines mehrfachen oder sonstigen schwerwiegenden Täuschungsversuches kann der Prüfling zudem exmatrikuliert werden. (§ 63 Abs. 5 Hochschulgesetz - HG -).

Die Abgabe einer falschen Versicherung an Eides statt wird mit Freiheitsstrafe bis zu 3 Jahren oder mit Geldstrafe bestraft.

Die Technische Universität Dortmund wird ggf. elektronische Vergleichswerkzeuge (wie z.B. die Software „turnitin") zur Überprüfung von Ordnungswidrigkeiten in Prüfungsverfahren nutzen.

Die oben stehende Belehrung habe ich zur Kenntnis genommen:

Official notification:
Any person who intentionally breaches any regulation of university examination regulations relating to deception in examination performance is acting improperly. This offense can be punished with a fine of up to EUR 50,000.00. The competent administrative authority for the pursuit and prosecution of offenses of this type is the Chancellor of TU Dortmund University. In the case of multiple or other serious attempts at deception, the examinee can also be unenrolled, Section 63 (5) North Rhine-Westphalia Higher Education Act (*Hochschulgesetz, HG*).

The submission of a false affidavit will be punished with a prison sentence of up to three years or a fine.

As may be necessary, TU Dortmund University will make use of electronic plagiarism-prevention tools (e.g. the "turnitin" service) in order to monitor violations during the examination procedures.

I have taken note of the above official notification:*

Dortmund, 07.08.2026 — Adolenko

Ort, Datum (place, date) — Unterschrift (signature)

*Please be aware that solely the German version of the affidavit ("Eidesstattliche Versicherung") for the Bachelor's/ Master's thesis is the official and legally binding version.